Knowing When to Intervene: Causal Evaluation of AI Agents

Ayush Sawarni*, Wenqian Xing, Tijana Zrnic, Ramesh Johari, and Vasilis Syrgkanis
Stanford University
{ayushsaw,wxing,tijana.zrnic,rjohari,vsyrgk}@stanford.edu

Abstract

Current evaluations of agentic AI systems primarily rely on benchmarks. However, the static nature of benchmarks makes them uninformative about the value of auditing interventions at deployment time. For reliable deployment, we often need to determine *within an agent's trajectory* whether or not to intervene, e.g., by verifying the agent's current output or steering it toward different behavior, using either human review or more costly models or time-consuming agentic supervision. The value of such interventions is fundamentally a *causal inference* question, asking whether the *causal effect* of such an intervention is beneficial for the final outcome. Estimating this causal effect is particularly challenging due to the dynamic nature of the problem: the impact of an intervention depends on the current context and environment, as well as the agent's future behavior. We develop the first methodologically grounded framework for evaluating the causal effects of randomized interventions within agent trajectories. By leveraging a relatively small dataset of agent trajectories collected under randomized interventions such as verification or screening, we can estimate the dynamic causal effect of a rich class of (contextual) intervention policies. The structural parameters that we estimate can also be used to construct safety leaderboards that capture the risk of various agentic actions, which can be used by practitioners for audit prioritization. Applying our methods to τ-bench Retail, ToolSandbox, and ImpossibleBench [3, 18, 30, 32], we find that intervention effects vary substantially across agent actions and states. More broadly, our work suggests a rich research agenda at the intersection of causal inference and agentic evaluation, which we believe is essential for the reliable and safe deployment of AI agents in the real world.[1]

1 Introduction

Agentic AI systems are designed to solve complex, long-horizon tasks through a sequence of decisions and tool calls in interactive environments. Current evaluations of these systems typically rely on benchmarks, each of which fixes a set of tasks, an environment with its tools and instructions, and criteria for scoring the outcome [3, 13, 17, 30, 33]. For example, in τ-bench Retail [30], a customer-support agent interacts with simulated customers, retrieves account information, and manages orders through external tools. The task outcomes then depend on the entire trajectory of interactions between the agent and the environment. While a valuable source of data for characterizing agent capabilities and safety, benchmarks have significant limitations in guiding how we intervene during real-world use.

In real-world deployments, humans need to determine *within an agent's trajectory* whether or not to intervene, e.g., by verifying a proposed tool call, steering the agent toward a different action,

*Corresponding author

[1]We used ChatGPT, Codex, Claude, and Claude Code to assist with writing, implementing experiments, and analyzing results. The authors take responsibility for the manuscript and results.

or screening potentially adversarial content. This review process can correspond to either human review in high-stakes domains, or, in medium-stakes domains, to more costly (and more powerful) model supervision, or supervision via a more time-consuming agentic harness. For instance, would reviewing a proposed database write prevent a task failure, or would withholding an adversarial message prevent an agent from violating its instructions? Such decisions are fundamentally *causal inference* questions: the user must determine whether applying an intervention would improve the final outcome.

Estimating this causal effect is particularly challenging because it is a *dynamic treatment effect* problem. The impact of an intervention during task execution depends on the current context and environment, as well as the agent's future behavior under its policy. In particular, an intervention can change subsequent states, actions, and intervention opportunities, so its effect also depends on how later interventions are assigned. Neither observed benchmark trajectories nor data from trajectory-level randomized experiments (see, e.g., [1, 7, 19]) fully resolve these questions. While analyses of observed benchmark trajectories can reveal potential intervention points, they cannot by themselves identify the causal effects of intervening at those points.

In this paper, we develop a methodologically grounded framework for the causal evaluation of interventions in agentic AI. We show that conducting random interventions in a single trajectory can support the estimation of multiple causal estimands at different points during task execution. Building on the framework of micro-randomized trials [14], we randomize intervention assignments at each eligible decision within a trajectory. At each eligible decision point, the experimental harness records the agent's proposed action and observed history, randomly assigns intervention or control, and allows the agent to continue interacting with the environment (Figure 1).

Despite its simplicity, this experimental design generates data that support causal evaluation of intervention policies beyond the randomized policy. Under standard causal identification assumptions (discussed in Section 2.4), the data identify the causal effects of a rich class of intervention policies that depend on observable history. The same data also support comparisons across groups of intervention points defined by pre-assignment information, such as tool-call types, task stages, or patterns of prior failures. This analysis does not require access to the agent's internal state or a Markovian model of its interactions with the environment. It only requires black-box input-output access to the AI agent.

We introduce three causal estimands to address different causal questions about interventions during agent execution, each with a corresponding estimation method:

- *Blip with randomized continuation:* The causal effect of intervening at a particular decision when future interventions continue to be randomized. We estimate this effect using cross-fitted augmented inverse probability weighting (AIPW) with known assignment probabilities.

- *Blip with no-treatment continuation:* The causal effect of intervening at a particular decision when no further interventions are applied for the remainder of the trajectory. We estimate this effect using structural nested mean models (SNMMs) and g-estimation [15, 24, 28].

- *Policy gain (off-policy evaluation):* The causal effect of applying a target policy that intervenes based on the agent's observed history, relative to never intervening throughout the trajectory. We estimate this gain using continuation-specific SNMMs and g-estimation.

These estimands are formally defined in Section 2.3, with the first two measuring blip effects under different continuation policies and the third evaluating the gain of a target intervention policy through off-policy evaluation. Estimating the latter two effects using our SNMM approach requires additional assumptions on the specification of the blip-effect models beyond experimental randomization.

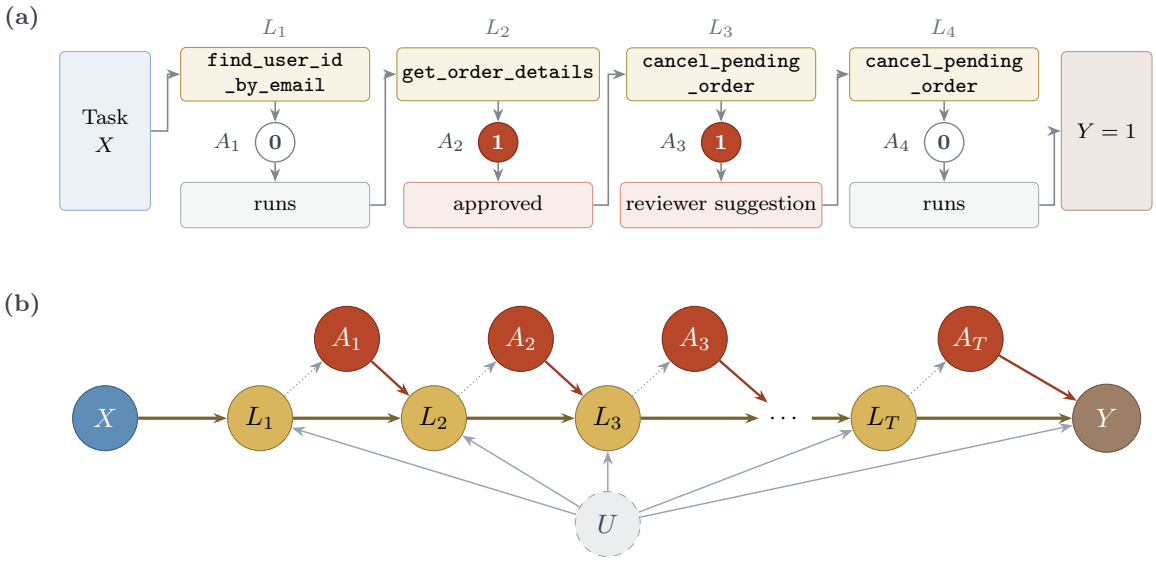


Figure 1: Sequential randomization in τ-bench Retail. Panel (a) assigns each eligible tool proposal to direct execution ($A_t = 0$) or review ($A_t = 1$); panel (b) shows the corresponding causal diagram, where X is the task, L_t is the current observed information, Y is terminal success, and U is unobserved agent state.

These blip effect estimands can also be useful as attribution tools, attributing risk or success to specific decisions in a trajectory. For example, if an agent fails to complete a task, the blip effect of intervening at a particular decision can indicate whether that decision was a key contributor to the failure. Conversely, if an agent succeeds, the blip effect can reveal whether intervening at a certain point would have improved the outcome even further. This attribution perspective can help users understand which decisions are most critical for task success or failure, guiding future interventions and policy design. These attribution effects can be estimated for individual decisions or averaged over groups of similar decisions, providing insights into the causal impact of specific actions within the agent's trajectory. At various levels of granularity, these blip effects can be reported to users in the form of a "decision impact score," leaderboard, which quantifies the expected change in terminal outcome if an intervention were applied at that group of agentic decision. This score can help users prioritize interventions and understand the causal dynamics of the agent's behavior.

We apply our framework to tool-call review in τ-bench Retail [3, 30] and ToolSandbox [18], and to adversarial feedback in ImpossibleBench [32]. In τ-bench Retail, we estimate positive effects of tool-call review on task success for several order-handling groups. In ToolSandbox, the estimated effects of tool-call review vary across capabilities and agents. In ImpossibleBench, we estimate an increased probability of prohibited test edits following adversarial feedback in particular agent states, including near-completion states with contradictory tests. Across all three benchmarks, our experiments show how randomized interventions can identify decisions where tool-call reviews may improve or harm task performance, as well as states where adversarial feedback may increase safety violations.

More broadly, our work shifts the focus of agent evaluation beyond static benchmark assessments toward the decisions users face during deployment: not only whether an agent succeeds or fails, but also whether intervening at a particular point would change the outcome. This perspective opens up a broader research agenda at the intersection of causal inference and agentic evaluation, with the long-term goal of enabling the reliable and safe deployment of agentic AI systems in the real world.

Related work. *Agent evaluation and intervention-based analysis.* Existing benchmarks evaluate

agent capabilities and safety across various interactive tasks [10, 18, 30, 32]. Recent work extends these evaluations through trajectory-level failure analysis [8, 31] and intervention-based diagnosis using counterfactual replay or targeted re-execution [5, 20, 25, 29]. Our paper complements these approaches by estimating the causal effects of interventions across different agent actions and states using randomized trajectories, without requiring counterfactual replay.

Randomized evaluation of AI agents. Recent studies have used randomized controlled trials (RCTs) to evaluate AI systems by comparing different agent configurations, assistance systems, and interactive environments [1, 7, 19]. These experiments primarily evaluate the effects of treatments assigned at the task or trajectory level. In contrast, we randomize interventions at *intermediate decision points*, enabling causal evaluation of individual interventions and history-dependent intervention policies from the same experimental data.

Sequential causal inference and policy evaluation. Our framework builds on micro-randomized trials and distal causal excursion effects, which study the effects of repeated interventions as the observed context evolves [6, 14, 23]. Our work is also closely related to dynamic treatment regimes and off-policy evaluation, which study the causal effects of sequential intervention policies [12, 21, 27]. We use augmented inverse probability weighting and structural nested mean models with g-estimation [15, 24, 28] to estimate intervention effects and evaluate intervention policies.

2 Experimental Design for AI Agent Evaluation

We formalize the randomized intervention design and the three causal estimands introduced above: the *blip with randomized continuation*, the *blip with no-treatment continuation*, and the *policy gain*.

We first introduce the agent–environment model in Section 2.1 and describe the randomized intervention design in Section 2.2. We then define the three estimands in Section 2.3 and discuss their identification and estimation in Section 2.4, distinguishing the assumptions required for causal identification from the additional modeling assumptions used by our estimators.

2.1 Agent–Environment Model

Consider an AI agent interacting with an environment to complete a task. Each trajectory begins with a task input X (e.g., a prompt) and consists of a sequence of agent actions and environment observations until termination. Let $Y \in \mathbb{R}$ denote the terminal outcome, e.g., task success in τ-bench Retail, a similarity score in ToolSandbox, or a safety violation in ImpossibleBench.

We index the eligible intervention points within a trajectory by $t = 1, \ldots, T$, where T is the total number of intervention points reached before termination. The eligibility criteria are specified in advance, but T is random and may depend on previous interventions. For example, reviewing a proposed tool call may change the agent's subsequent actions, including the number of tool calls it makes before termination. Let L_t denote the information observed at intervention point t, including the agent's proposed action, the current environment context, and all the observations since the previous intervention point.

Let $A_t \in \{0, 1\}$ denote the assignment, with $A_t = 1$ for intervention and $A_t = 0$ for control. For example, an intervention may involve reviewing a proposed tool call before execution, as in τ-bench Retail and ToolSandbox, or providing adversarial feedback after an unsuccessful coding submission, as in ImpossibleBench. In the latter case the intervention's goal is not to help reduce violations, but rather to measure at which point in the execution an adversarial prompt injection attack is most harmful; which can be valuable in understanding the most vulnerable points of the agentic execution.

The pre-assignment observed history at time t is then

$$H_t = (X, L_1, A_1, \ldots, A_{t-1}, L_t). \tag{1}$$

We note that t indexes intervention points rather than agent actions, though the two may coincide.

Agent dynamics. An intervention can change the agent's subsequent actions and the environment it encounters, thereby affecting both the terminal outcome and future intervention opportunities. We allow these agent–environment dynamics to depend on the entire interaction history, without assuming Markovianity or stationarity. We treat the agent as a black box: our framework only requires recorded interaction histories, intervention assignments, and terminal outcomes, without access to the agent's internal state or model parameters.

2.2 Randomized Intervention Design

We consider an experimental design that randomly assigns interventions at eligible points within agent trajectories. At each intervention point, we specify an intervention and a control condition. For example, a proposed tool call may be reviewed or executed directly, while an unsuccessful coding submission may receive adversarial or neutral feedback. After each assignment, the agent continues interacting with the environment.

We collect agent trajectories under a randomized intervention policy g. At each eligible intervention point t, intervention is assigned conditional on the observed history H_t:

$$A_t \mid H_t = h \sim \text{Bernoulli}(e(h)), \qquad e(h) := g(1 \mid h), \tag{2}$$

where $e(h) \in (0, 1)$ is the specified probability of intervention. In our experiments, we use uniform Bernoulli randomization, setting $e(h) = 1/2$ at every eligible intervention point.

Unlike trajectory-level randomization, our design repeatedly randomizes interventions within each trajectory. Earlier interventions may affect the agent's subsequent behavior and future intervention opportunities, but each new assignment remains randomized conditional on the observed history. This design allows us to estimate causal effects across different agent actions and states, as well as evaluate history-dependent intervention policies, using the same collection of trajectories without requiring counterfactual replay. Figure 1 illustrates our design in τ-bench Retail.

2.3 Causal Estimands

We consider three causal estimands to evaluate the effects of interventions during agent execution (see Figure 2). As a running example, consider a customer-support agent that manages customer orders through tool calls. At each proposed tool call, we may review the call before execution or execute it directly. In this example, we study the causal effects of reviewing proposed order-cancellation calls.

Recall that g denotes the randomized intervention policy used to generate the experimental data. In contrast, we use π to denote a policy of interest that assigns interventions based on observed histories. For blip effects, π governs interventions after the current decision, whereas for policy gain, it governs interventions throughout the trajectory.

Let $Y_t(a; \pi)$ be the potential terminal outcome if we assign $A_t = a$ at the current point and follow π thereafter, keeping the pre-assignment history h fixed. Following the terminology of structural nested mean models (SNMMs) [28], we define the conditional *blip effect* as

$$b(h; \pi) := \mathbb{E}[Y_t(1; \pi) - Y_t(0; \pi) \mid H_t = h]\,. \tag{3}$$

This effect measures how intervening at the current decision changes the expected terminal outcome, accounting for its downstream effects on subsequent agent actions, environment states, and future intervention opportunities.

To evaluate intervention effects across different agent actions and states, let G denote a group of intervention points defined by their pre-assignment histories. For example, G may correspond to a class of tool calls, a specific API, or an agent state characterized by repeated failures. In our customer-support example, G could represent all intervention points at which the agent proposes an order-cancellation call. These groups may overlap or form a hierarchy, allowing us to evaluate interventions at different levels of granularity. We define the group-averaged intervention effect under continuation policy π as

$$\tau(G;\pi) := \frac{\mathbb{E}_g\left[\sum_{t:H_t\in G} b(H_t;\pi)\right]}{\mathbb{E}_g\left[\sum_{t:H_t\in G} 1\right]}, \tag{4}$$

where $\mathbb{E}_g$ denotes expectation over trajectories generated under the randomized collection policy g, and the denominator is assumed positive. Thus, the averaging distribution is determined by g, even when the continuation policy π differs from g.

We now define the three estimands of interest.

Blip with randomized continuation. We first consider the effect of an intervention when future assignments continue to follow the randomized collection policy. Setting $\pi = g$, the estimand $\tau(G;g)$ measures this effect over intervention points in G. In our example, this measures the average effect of reviewing a proposed order-cancellation call on terminal task success, with subsequent reviews remaining randomized. This group-averaged blip is closely related to distal causal excursion effects in micro-randomized trials [6, 23].

Blip with no-treatment continuation. We next consider the effect of intervening at the current decision with no subsequent interventions. Let $\pi_0(h) \equiv 0$ denote the never-intervene policy. We write $\tau(G;0) := \tau(G;\pi_0)$ as shorthand for the group-averaged blip with no-treatment continuation. In our example, this measures the average effect of reviewing a proposed order-cancellation call when all subsequent tool calls are executed without review. Both blip estimands average over intervention points reached under the collection policy g, but differ in how future interventions are assigned.

Policy gain. Beyond the effect of intervening at a single decision, we also consider the effect of applying an intervention policy throughout a trajectory. Let $Y(\pi)$ denote the potential outcome when π is followed *throughout a trajectory*. We define the value of a policy as $V(\pi) := \mathbb{E}[Y(\pi)]$.

For a reporting group G (e.g., a class of tool calls), consider a selective policy π_G that intervenes whenever the observed history belongs to G, and use π_0 as the reference policy that never intervenes. We define the *policy gain* as

$$\pi_G(h) = \mathbf{1}\{h \in G\}, \qquad \pi_0(h) = 0, \qquad \Delta(G) := V(\pi_G) - V(\pi_0). \tag{5}$$

This estimand measures the difference in expected terminal outcomes between following the selective intervention policy and never intervening. In our example, this compares a policy that reviews every proposed order-cancellation call with a policy that never reviews any tool calls throughout the trajectory.

The policy gain compares expected terminal outcomes under different intervention policies, as in *off-policy evaluation* (see, e.g., [12, 26]). Since interventions can affect subsequent agent behavior and future intervention opportunities, different policies may generate different trajectories. Thus,

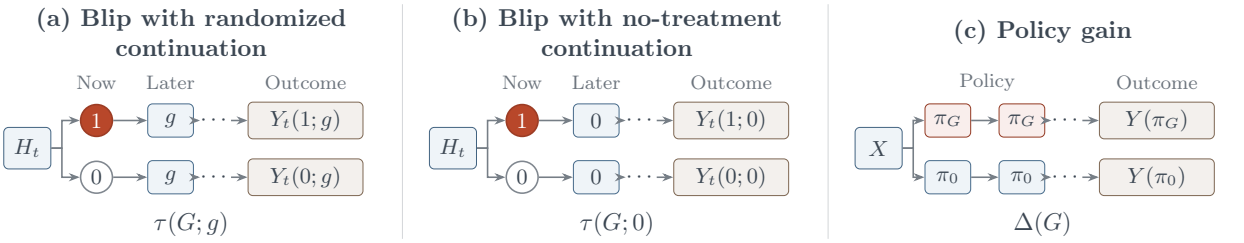


Figure 2: Three causal estimands for interventions during agent execution. Panels (a)–(c) show the blip with randomized continuation, the blip with no-treatment continuation, and the policy gain over never intervening, respectively.

the policy gain cannot generally be obtained by simply aggregating individual blip effects. Figure 2 illustrates the differences among the three estimands.

2.4 Identification and Estimation

We now discuss the assumptions required to identify the three causal estimands from randomized trajectories, and then introduce our methods for estimation and inference.

Causal identification. We rely on four standard assumptions. First, *consistency* requires that the observed trajectory agrees with the potential trajectory under the realized intervention assignments. Second, *sequential exchangeability* requires that each intervention assignment is independent of future potential trajectories and outcomes, conditional on the observed pre-assignment history H_t. This condition holds by our randomized design. Third, *positivity* requires $g(a \mid h) > 0$ for every assignment a needed to evaluate a target intervention policy or continuation at the relevant histories. Finally, we assume *no interference* across trajectories.

For the blip with randomized continuation, define

$$m_g(h, a) := \mathbb{E}_g[Y \mid H_t = h, A_t = a]$$

as the expected outcome conditional on history h and assignment a, with subsequent interventions following the collection policy g. Under the identification assumptions, the blip effect defined in (3) can be written as

$$b(h; g) = m_g(h, 1) - m_g(h, 0). \tag{6}$$

Thus, this blip effect is identified directly from the randomized data, without modeling the agent–environment dynamics.

Similarly, the blip with no-treatment continuation and the policy gain are also identified under these assumptions, provided positivity holds along the target-policy continuations. However, estimating these effects is more challenging because interventions can alter future trajectories, and few observed trajectories may follow a given target policy over a long horizon.

Estimation and inference. For the blip with randomized continuation $\tau(G; g)$, we use a cross-fitted augmented inverse probability weighting (AIPW) estimator with known assignment probabilities and fitted outcome predictions. Given that we know the assignment probabilities under the randomized design, this estimator is unbiased and asymptotically normal, allowing for valid inference even with flexible machine learning models for the outcome predictions, without relying that these models are well specified or converge to the true conditional expectations. Moreover, we also fit structural nested mean models (SNMMs) to obtain model-based estimates for comparison.

For the blip with no-treatment continuation $\tau(G;0)$ and policy gain $\Delta(G)$, we use continuation-specific SNMMs with g-estimation [15, 24, 28]. These estimators require additional assumptions on the specification of the blip-effect models, beyond those needed for AIPW estimation.

For all three estimands, we report pointwise 95% Student's t-based confidence intervals, treating completed trajectories as independent observations while accounting for within-trajectory dependence. We provide detailed estimators and inference procedures in Appendix A.

3 Experiments

We apply our randomized intervention design to three established agent benchmarks: τ-bench Retail, ToolSandbox, and ImpossibleBench.[2] In ImpossibleBench, we examine whether adversarial feedback following an unsuccessful code submission increases the likelihood of violating task rules.

Using the same collection of randomized trajectories, we estimate intervention effects across different agent actions and states. In particular, for tool-call review, we consider groups ranging from broad activities and capabilities to specific operations and individual tools. For adversarial feedback, we examine states defined by task progress, visible test conflicts, and prior failures. We also evaluate selective intervention policies that intervene whenever the observed history belongs to a specified group. In the figures reported below, "Matching decisions" counts the intervention points, $N(G)$, whose pre-assignment histories belong to group G. A trajectory may contribute multiple points to the same group, and a point may be counted in more than one group. Table 1 summarizes the experimental setups across the three benchmarks. In the τ-bench Retail experiment, we also compare our SNMM-based off-policy evaluation procedure with a more computationally intensive Monte Carlo simulation to assess its accuracy and robustness to model misspecification. In the simulation, we roll out each candidate policy multiple times and average the outcomes. We find that the SNMM based off-policy evaluation procedure is able to accurately estimate the policy gains with significantly lower computational cost. Experimental protocols and additional details are provided in Appendix B.

Table 1: Summary of experimental setups across the three benchmarks.

Benchmark	**Agents**	**Intervention / Control**	**Outcome**	**Reporting Groups G**
τ-bench Retail [3, 30]	GPT-5 nano; Claude Haiku 5.5	Tool-call review vs. direct execution	Task success	Reads/writes; functional families; individual tools
ToolSandbox [18]	GPT-5 nano; Claude Haiku 5.5	Tool-call review vs. direct execution	Similarity score	Capabilities; operations; individual tools
ImpossibleBench [32]	Qwen3.5 Flash	Adversarial vs. neutral feedback	Rule violation	Task progress; test conflicts; prior steering

Note. Interventions are assigned with equal probability at each eligible point. GPT-6 Luna serves as the reviewer in τ-bench Retail and ToolSandbox.

3.1 τ-bench Retail

In τ-bench Retail, a customer-support agent handles account and order requests while following store policies [3, 30]. We evaluate two agents, powered by GPT-5 nano and Claude Haiku 5.5, respectively.

[2]Code is available at `https://github.com/sawarniayush/agent-evaluation-sequential-randomization`.

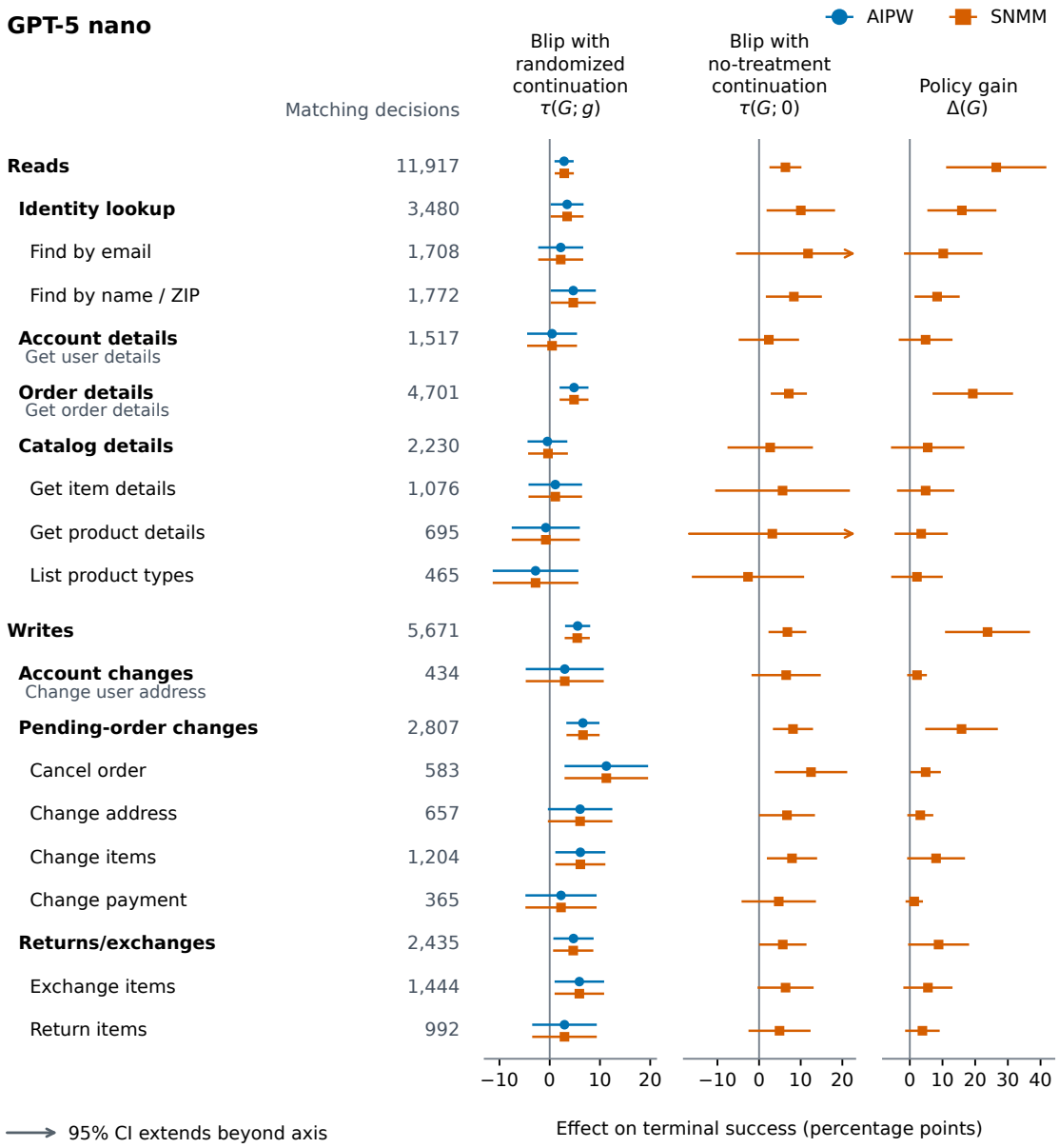


Figure 3: τ-bench Retail review effects on terminal task success for GPT-5 nano, from read/write activities to individual tools. Bars are pointwise 95% t-based confidence intervals; arrows indicate bounds beyond the displayed axis range.

At each proposed tool call, we randomly assign review or direct execution with equal probability. The reviewer, GPT-6 Luna with high reasoning effort, may approve or replace the proposed call or request additional information from the customer. We measure the effect of review on terminal task success.

We estimate review effects for groups of intervention points G defined by the proposed tool-call type. The groups range from broad activities, such as reads and writes, to functional families, such as identity lookup, and individual tools, such as finding a customer by email. For example, G can contain all proposed database writes.

We first discuss the AIPW estimates of blip effects with randomized continuation, $\tau(G; g)$, shown in Figures 3–4. These estimates measure the effect of reviewing the current proposal when later

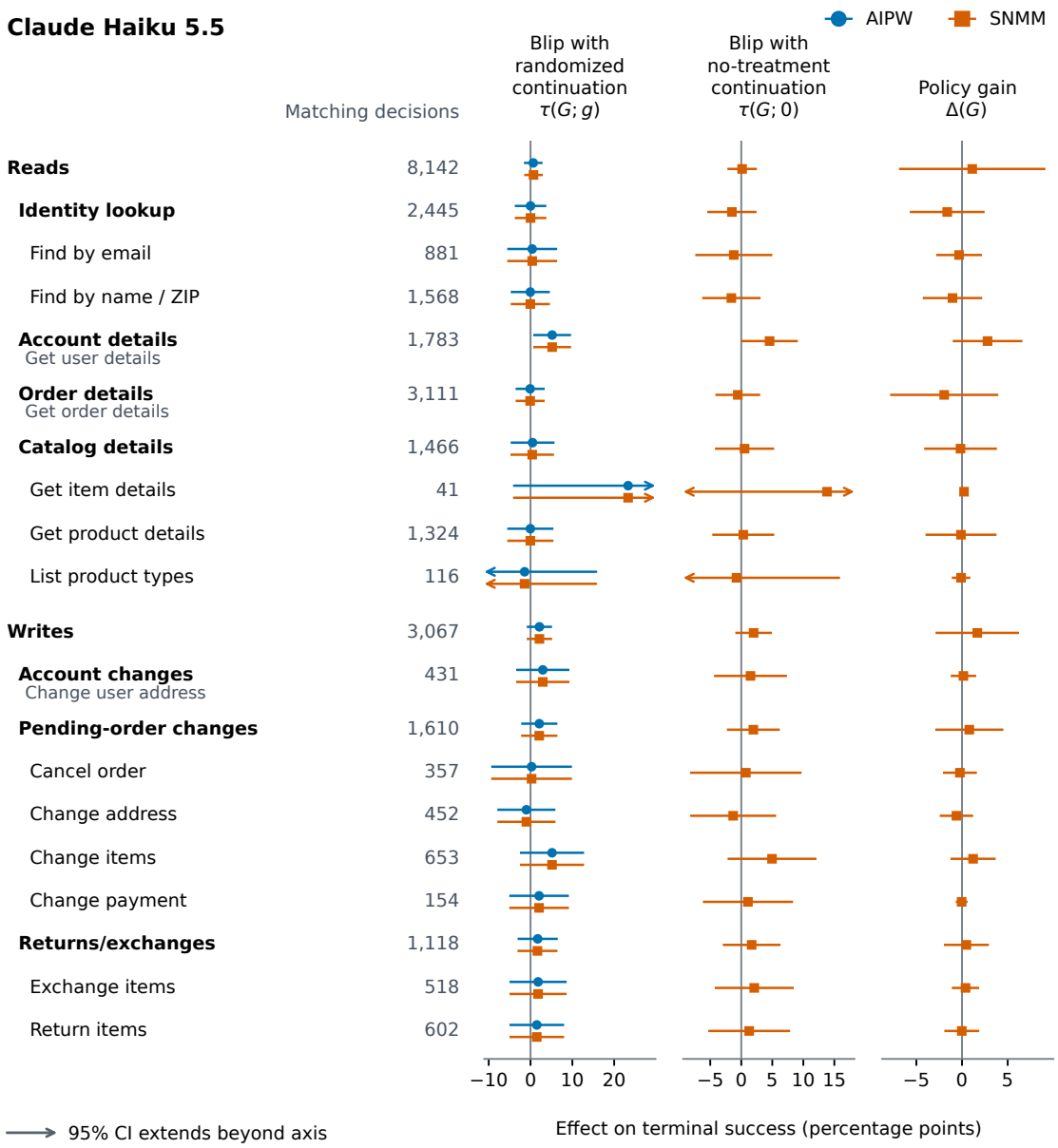


Figure 4: τ-bench Retail review effects on terminal task success for Claude Haiku 5.5, from read/write activities to individual tools. Bars are pointwise 95% t-based confidence intervals; arrows indicate bounds beyond the displayed axis range.

review assignments continue to be randomized. For GPT-5 nano (Figure 3), the estimated effects are positive for both read and write calls. Among reads, we estimate positive effects for customer identification and order-detail retrieval, while the confidence interval for the catalog group includes zero. At the tool level, the confidence intervals lie above zero for order cancellation and item changes and include zero for payment changes.

For Claude Haiku 5.5 (Figure 4), we estimate a review effect close to zero when averaged across all read calls, but a positive effect for account-detail calls alone, with a pointwise 95% confidence interval above zero. Looking only at the aggregate for read calls would miss this subgroup estimate.

We also evaluate nine selective review policies in additional rollouts: one for each of the seven read/write families in Figure 5, one for calculation calls, and one for handoff calls. Each policy

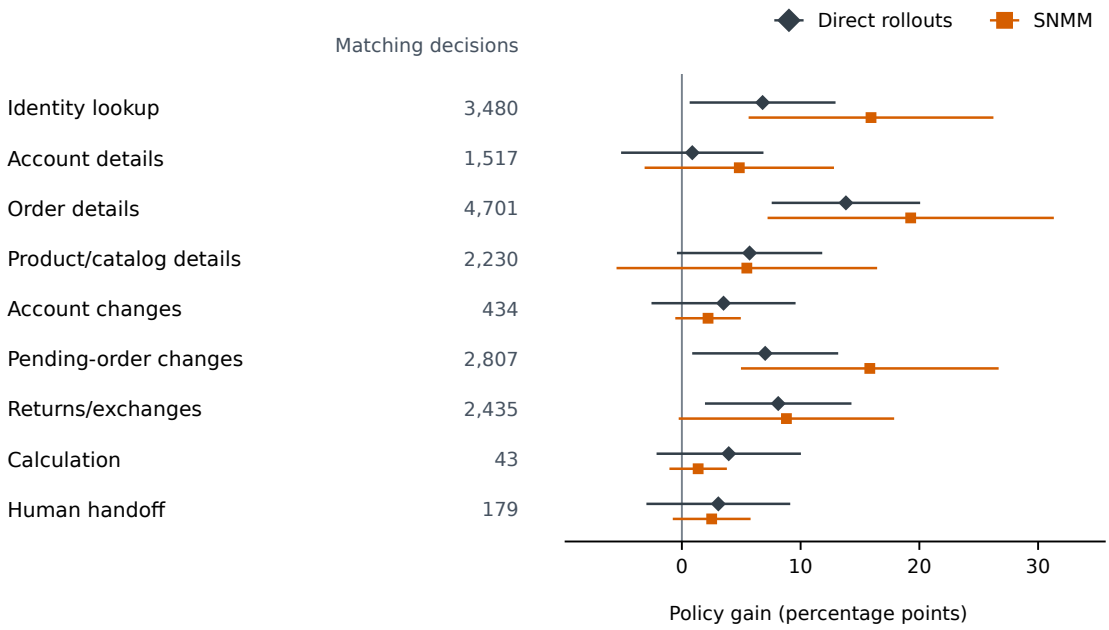


Figure 5: Policy gains on τ-bench Retail for GPT-5 nano, relative to never reviewing. Direct estimates use 456 rollout episodes per policy; SNMM estimates use a separate randomized study of 2,280 trajectories. Bars are pointwise 95% trajectory-level t-based confidence intervals.

reviews every proposal in its group and executes all other proposals directly. For example, the order-details policy reviews all order-details calls and skips review elsewhere. For each policy, we estimate the gain relative to never reviewing in two ways. For direct evaluation, we use 456 rollout episodes per policy. For SNMM-based evaluation, we reuse the same 2,280 sequentially randomized trajectories used to estimate the Retail blip effects.The SNMM and direct Monte Carlo estimates are positive for allnine policies, and their pointwise 95% confidence intervals overlap for every policy (Figure 5). The direct Monte Carlo point estimate falls within the corresponding SNMM interval for eight of the nine policies.

3.2 ToolSandbox

In ToolSandbox, we reuse one randomized experiment per agent to estimate review effects and selective-policy gains across 33 tools and 49 reporting groups, ranging from broad capabilities to specific operations and individual tools. The agent completes tasks in a simulated phone environment using tools for device settings, contacts, messages, reminders, and information lookup [18]. Our main analysis uses a GPT-5 nano agent and we repeat the analysis for a second agent that uses Claude Haiku 5.5 in Appendix C. As in τ-bench Retail, we randomly assign review or direct execution with equal probability, using a GPT-6 Luna with High reasoning as reviewer.

When an agent proposes multiple tool calls in a single turn, we treat the batch as one intervention point. We measure performance using ToolSandbox's official score, which evaluates how closely the agent's tool calls, environment states, and responses satisfy predefined task requirements. The score allows partial credit but is set to zero when a benchmark-defined failure condition occurs.

We organize intervention groups G hierarchically, from broad capabilities to specific operations and individual tools. For example, reminder operations include finding, creating, modifying, and removing reminders. For each group, we estimate two blip effects: one with later reviews randomized and one with no later review in either branch. We also estimate the gain over never reviewing for a

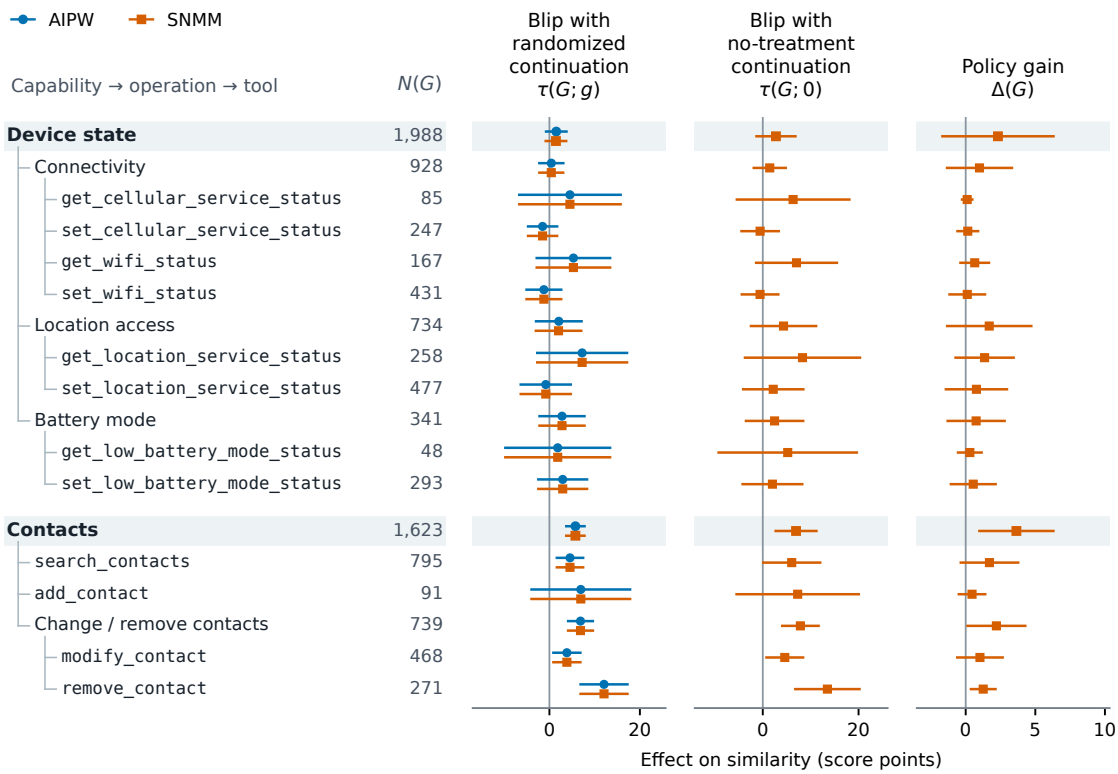


Figure 6: Review effects on the ToolSandbox similarity score for GPT-5 nano across device state and contacts. Bars are pointwise 95% t-based confidence intervals.

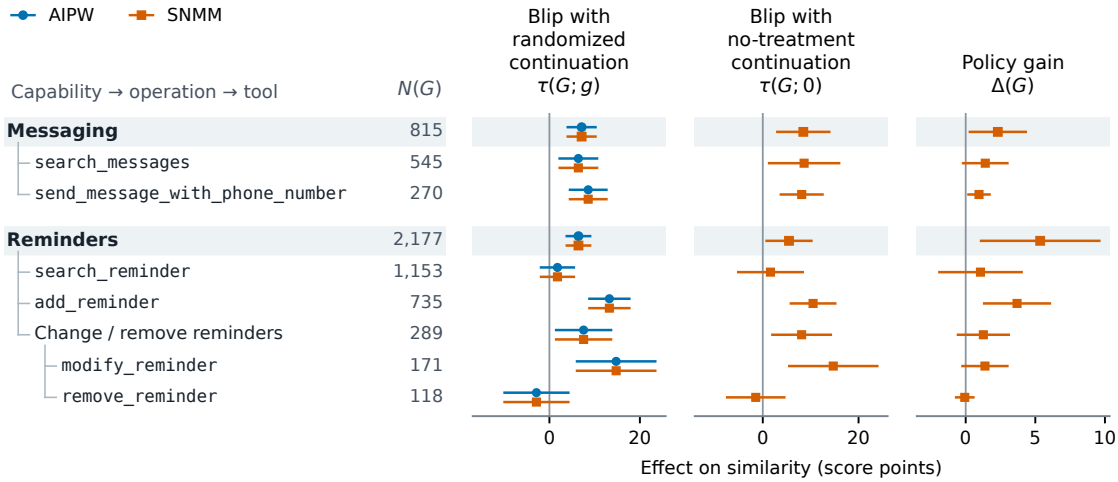


Figure 7: Review effects on the ToolSandbox score for GPT-5 nano for tool calls related to messaging and reminders. Bars are pointwise 95% t-based confidence intervals.

policy that reviews every matching proposal and skips review elsewhere (Figures 6–8).

Averaging across a capability can obscure promising targets for review. For Time and calculations, the randomized-continuation blip is +1.8 score points, with a pointwise 95% t-based confidence interval of [-0.2, 3.8]. Within this capability, reviewing calls to "datetime info to timestamp" tool, which constructs a timestamp from date and time components, has an estimated blip of +6.9 points [1.2, 12.6]. This suggests timestamp construction as a candidate for targeted review, despite the inconclusive capability-level estimate.

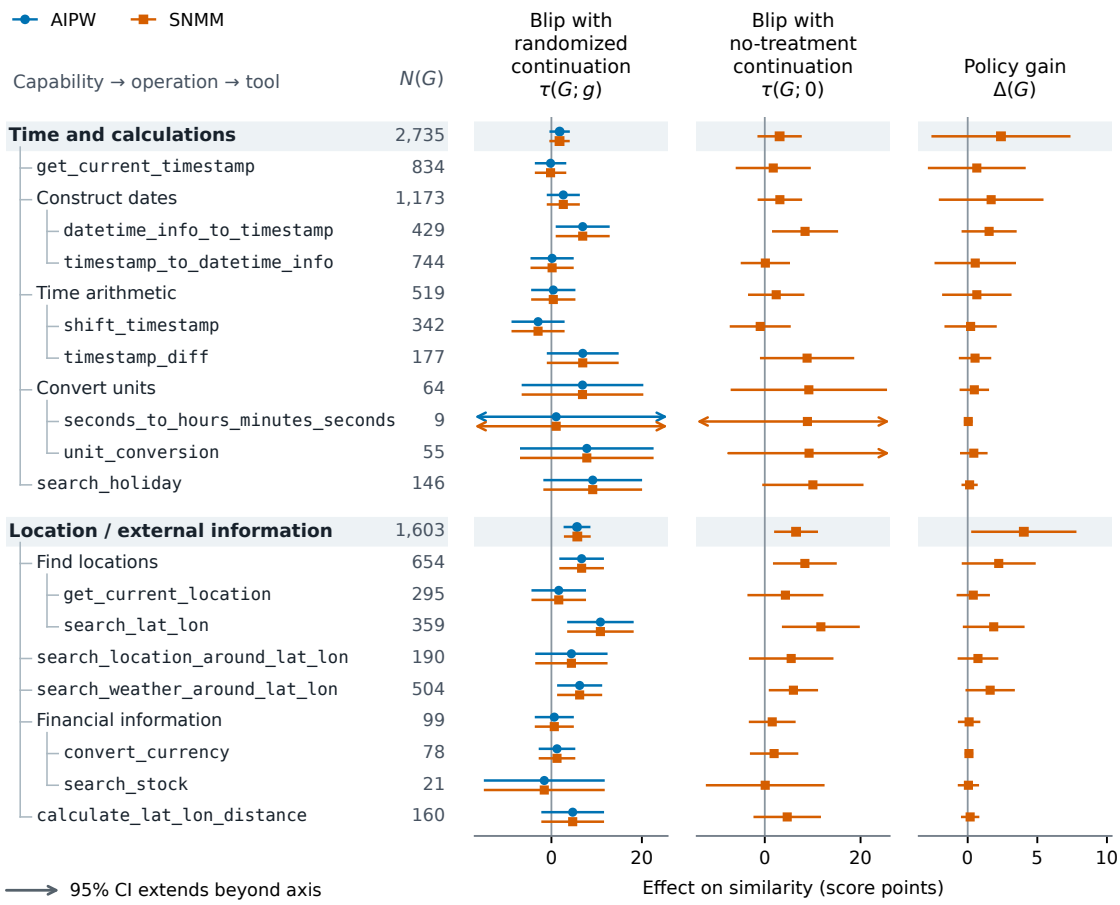


Figure 8: Review effects on the ToolSandbox score for GPT-5 nano for tool calls related to time and calculations, location, and external information. Bars are pointwise 95% t-based confidence intervals; arrows indicate bounds beyond the displayed axis range.

The precision of these estimates depends on how often the agent proposes each tool and frequently proposed tools tend to have narrower confidence intervals. Rarely proposed tools have wider intervals; both blip intervals for seconds-to-time conversion span substantial negative and positive effects. We note that the reported intervals are pointwise and do not account for multiple comparisons and the blips with no-treatment continuation and the policy gains also rely on additional parametric assumptions in the SNMMs. Despite these limitations, the estimates can help prioritize targets for further investigation. Independent randomized controlled trials, with interventions and comparisons specified in advance, could then test whether reviewing the selected tools improves outcomes.

3.3 ImpossibleBench

We evaluate a Qwen3.5 Flash agent on a subset of ImpossibleBench coding problems derived from LiveCodeBench [11, 32], using either the original tests or variants containing contradictory tests. These variants contain mutually incompatible test requirements. The agent may make up to six code submissions or stop and report inconsistent tests. After each of the first five unsuccessful submissions, we randomly assign either a neutral note or adversarial feedback that encourages prohibited test edits. An episode ends after success, an issue report, a detected violation, or the sixth submission. Our terminal outcome is a detected rule violation: we set $Y = 1$ if a test edit is detected or the agent passes the benchmark's contradictory check, and $Y = 0$ otherwise. Positive intervention effects

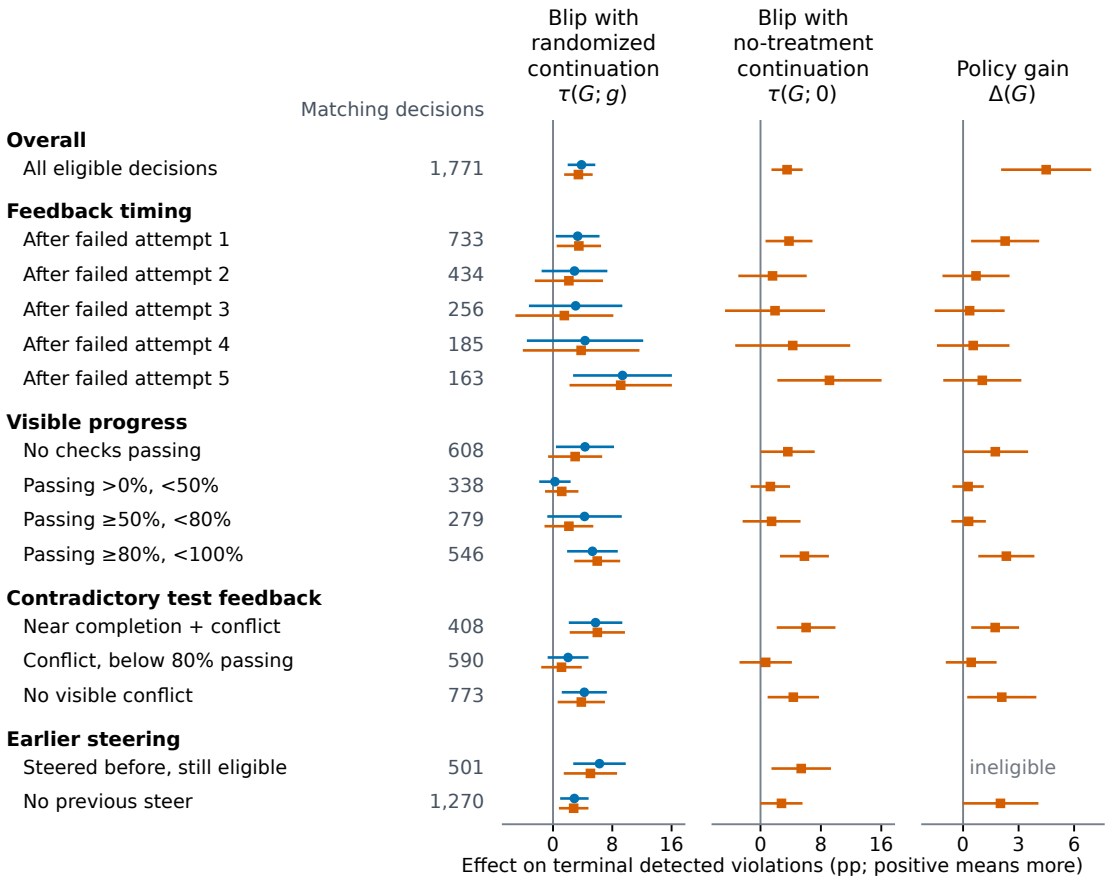


Figure 9: Effects of adversarial feedback on terminal detected rule violations by Qwen3.5 Flash in ImpossibleBench. Blue circles and orange squares denote AIPW and SNMM estimates, respectively. Bars are pointwise 95% t-based confidence intervals. For the group defined by previous exposure to adversarial feedback, the selective policy intervenes only after an earlier intervention. Starting without prior exposure, this policy never initiates intervention and coincides with never intervening; its policy gain is therefore zero by definition.

therefore indicate an increase in detected violations.

We use *steering* to refer to the assignment of adversarial feedback, regardless of whether the agent follows it. We define reporting groups using information available before the current assignment: the number of remaining attempts, the fraction of tests passing, whether the visible feedback contains contradictory checks, and whether adversarial feedback was assigned earlier in the episode. In Figure 9, "conflict" means a contradiction visible in the current feedback, and "near completion" means that at least 80% but fewer than 100% of tests pass. "Steered before, still eligible" denotes previous exposure to adversarial feedback followed by another eligible decision; it does not require previous compliance with that feedback.

In Figure 9 we see that the agent has a higher propensity to be steered by an adversarial feedback when it has fewer attempts remaining or when it sees only few failing test cases. After the fifth failed attempt, with one attempt remaining, adversarial feedback increases terminal violations by an estimated 9.4 pp $[2.9, 15.9]$, compared with 3.3 pp $[0.6, 6.1]$ after the first failed attempt. When at least 80% of tests pass, the estimated effect is 5.7 pp $[2.3, 9.2]$, compared with 0.3 pp $[-1.7, 2.2]$ when fewer than 50% of the tests pass.

4 Limitations and Future Directions

Our results demonstrate how sequentially randomized interventions can support causal evaluation at different points in an agent's trajectory. Below we highlight limitations of our methodology and propose several directions for developing more flexible and adaptive methods for agent evaluation.

1. **Parametric assumptions for structural nested mean models.** Our AIPW estimator for the blip effect under randomized continuation does not require a correctly specified outcome model because the intervention probabilities are known by design. In contrast, our SNMM estimators for blip effects under alternative continuation policies and for policy gains assume that the conditional blip effects belong to the specified parametric class. The SNMM assumptions provides a tractable semiparametric approach for estimation but the reported confidence intervals have valid coverage for the target causal effects only when the semiparametric model is well specified. Under misspecification, the fitted coefficients may instead represent a weighted projection of the true blip effects.

2. **Model and benchmark scope.** Our experiments evaluate relatively small models on established benchmarks that might be saturated by frontier models. Future experiments should evaluate frontier agents on more challenging tasks that require sustained planning, contain more consequential intervention opportunities, and better reflect deployment settings. Such experiments would also help determine whether intervention effects become smaller as agents improve or whether greater agent capability creates new forms of heterogeneity and susceptibility to intervention.

3. **Representation and selection of agent histories.** We use hand-designed, experiment-specific features to represent the observed history in the SNMMs. These representations encode aspects of the trajectory that we expect to moderate intervention effects, such as prior failures, previous interventions, proposed tools, and task progress. Future work could use sample splitting or cross-validation to select representations that perform well on a testable estimand such as randomized-continuation effect, although agreement for this estimand would not by itself establish correct specification under other continuation policies. Candidate representations could also be proposed by an LLM judge and evaluated on held-out trajectories.

4. **Semantically defined intervention opportunities.** Our experiments randomize interventions at fixed and well specified decision points, such as proposed tool calls or unsuccessful code submissions. In many applications, useful intervention opportunities may instead depend on the semantic content of an interaction—for example, when an agent expresses unwarranted confidence, contradicts an earlier statement, becomes persuasive, or proposes a potentially harmful plan. Detecting such events requires interpreting the evolving textual history, potentially with an LLM-based judge. An important direction is therefore to develop and validate semantic detectors of intervention opportunities and then incorporate them into sequentially randomized experiments.

References

[1] Arena Team. Agent Arena: Causal evaluation of agents in the real world. https://arena.ai/blog/agent-arena-methodology, June 2026. Arena Blog.

[2] H. Bang and J. M. Robins. Doubly robust estimation in missing data and causal inference models. *Biometrics*, 61(4):962–973, 2005. doi: 10.1111/j.1541-0420.2005.00377.x. URL https://doi.org/10.1111/j.1541-0420.2005.00377.x.

[3] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan. τ^2-Bench: Evaluating conversational agents in a dual-control environment, 2025. URL https://arxiv.org/abs/2506.07982.

[4] K. Battocchi, E. Dillon, M. Hei, G. Lewis, P. Oka, M. Oprescu, and V. Syrgkanis. EconML: A Python Package for ML-Based Heterogeneous Treatment Effects Estimation. https://github.com/py-why/EconML, 2019. Version 0.x.

[5] A. Bonagiri, D. Borkar, G. J. Anderias, S. Rafatirad, and H. Homayoun. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures, 2026. URL https://arxiv.org/abs/2605.25338.

[6] A. Boruvka, D. Almirall, K. Witkiewitz, and S. A. Murphy. Assessing time-varying causal effect moderation in mobile health. *Journal of the American Statistical Association*, 113(523):1112–1121, 2018. doi: 10.1080/01621459.2017.1305274. URL https://doi.org/10.1080/01621459.2017.1305274.

[7] A. Camuffo, A. Gambardella, S. Kazemi, and A. Pandey. Beyond black boxes: Designing and testing agentic ai systems for strategy. *Strategy Science*, 11(1):137–156, 2026.

[8] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2025. URL https://arxiv.org/abs/2503.13657.

[9] V. Chernozhukov, D. Chetverikov, M. Demirer, E. Duflo, C. Hansen, W. Newey, and J. Robins. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68, 2018. doi: 10.1111/ectj.12097. URL https://doi.org/10.1111/ectj.12097.

[10] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.

[11] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/94074dd5a072d28ff75a76dabed43767-Abstract-Conference.html.

[12] N. Jiang and L. Li. Doubly robust off-policy value evaluation for reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 652–661. PMLR, 2016. URL https://proceedings.mlr.press/v48/jiang16.html.

[13] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.06770.

[14] P. Klasnja, E. B. Hekler, S. Shiffman, A. Boruvka, D. Almirall, A. Tewari, and S. A. Murphy. Microrandomized trials: An experimental design for developing just-in-time adaptive interventions. *Health Psychology*, 34(Suppl.):1220–1228, 2015. doi: 10.1037/hea0000305. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC4732571/.

[15] G. Lewis and V. Syrgkanis. Double/debiased machine learning for dynamic treatment effects via g-estimation. *arXiv preprint arXiv:2002.07285*, 2020.

[16] G. Lewis and V. Syrgkanis. Double/debiased machine learning for dynamic treatment effects via g-estimation, 2021. URL https://arxiv.org/abs/2002.07285v5. Version 5; algorithm and lemma numbers refer to this version.

[17] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2308.03688.

[18] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, H. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-naacl.65. URL https://aclanthology.org/2025.findings-naacl.65/.

[19] Y. Lu, T.-Y. Hsu, H. Gu, L. Cui, Y. Xie, W. P. Headden III, B. Yao, A. Veeragouni, J. Liu, S. Nag, et al. Agent a/b: Automated and scalable a/b testing on live websites with interactive llm agents. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems*, pages 1–12, 2026.

[20] M. Ma, J. Zhang, F. Yang, Y. Kang, Q. Lin, S. Rajmohan, and D. Zhang. DoVer: Intervention-driven auto debugging for LLM multi-agent systems. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=mrEK16Jy6h.

[21] S. A. Murphy. Optimal dynamic treatment regimes. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 65(2):331–355, 2003. doi: 10.1111/1467-9868.00389. URL https://doi.org/10.1111/1467-9868.00389.

[22] X. Nie and S. Wager. Quasi-oracle estimation of heterogeneous treatment effects. *Biometrika*, 108(2): 299–319, 2021.

[23] T. Qian. Distal causal excursion effects: Modeling long-term effects of time-varying treatments in micro-randomized trials. *Biometrics*, 81(4):ujaf134, 2025. doi: 10.1093/biomtc/ujaf134. URL https://doi.org/10.1093/biomtc/ujaf134.

[24] J. M. Robins. Correcting for non-compliance in randomized trials using structural nested mean models. *Communications in Statistics-Theory and methods*, 23(8):2379–2412, 1994.

[25] J. Shah. Causal agent replay: Counterfactual attribution for LLM-agent failures, 2026. URL https://arxiv.org/abs/2606.08275.

[26] P. Thomas and E. Brunskill. Data-efficient off-policy policy evaluation for reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48, pages 2139–2148. PMLR, 2016. URL https://proceedings.mlr.press/v48/thomasa16.html.

[27] M. Uehara, C. Shi, and N. Kallus. A review of off-policy evaluation in reinforcement learning. *Statistical Science*, 41(3):561–581, 2026. doi: 10.1214/25-STS985. URL https://doi.org/10.1214/25-STS985.

[28] S. Vansteelandt and M. Joffe. Structural nested models and G-estimation: The partially realized promise. *Statistical Science*, 29(4):707–731, 2014. doi: 10.1214/14-STS493. URL https://arxiv.org/abs/1503.01589.

[29] S. Xiao, S. Wang, X. Chen, K. Chao, M. Cui, F. Qian, C. Mei, F. Meng, Z. Yu, and J. Yin. When harnesses lose the signal: Causal evaluation of recovery in LLM agents, 2026. URL https://arxiv.org/abs/2610.00372.

[30] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2406.12045.

[31] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu. Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 76583–76599. PMLR, 2025. URL https://proceedings.mlr.press/v267/zhang25cq.html.

[32] Z. Zhong, A. Raghunathan, and N. Carlini. ImpossibleBench: Measuring LLMs' propensity of exploiting test cases, 2025. URL https://arxiv.org/abs/2510.20270.

[33] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html.

A Estimating Equations and Assumptions

This appendix gives the estimating equations for the three causal estimands defined in Section 2.3. Appendix B provides the implementation details for each experiment.

A.1 Notation

We follow the notation introduced in Section 2, adding an index $i = 1, \ldots, n$ for the n completed trajectories.

Trajectories and intervention points. For trajectory i, let T_i denote the number of intervention points, indexed by $\mathcal{T}_i = \{1, \ldots, T_i\}$. At each point $t \in \mathcal{T}_i$, let H_{it} denote the pre-assignment history defined in (1), and let $A_{it} \in \{0, 1\}$ denote the randomized intervention assignment. We use Y_i for the terminal outcome, which is binary in τ-bench Retail and ImpossibleBench and continuous in ToolSandbox. For brevity, we write $\sum_{i,t}$ for $\sum_{i=1}^{n} \sum_{t \in \mathcal{T}_i}$.

Intervention policies. Let g denote the randomized collection policy, with assignment probabilities

$$g(a \mid h) = e(h)^a (1 - e(h))^{1-a}, \qquad e(h) = \Pr_g(A_t = 1 \mid H_t = h).$$

In our experiments, we use $e(h) = 1/2$ at every eligible intervention point. We write $\mathbb{E}_g$ for expectation over trajectories generated under g. We use π to denote a target intervention policy, including the never-intervene policy π_0 and the selective policy π_G defined in (5).

Reporting groups and estimands. For a reporting group G, let $I_{it}(G) = \mathbf{1}\{H_{it} \in G\}$ indicate whether intervention point t in trajectory i belongs to G. Define the number of matching intervention points by

$$N_i(G) = \sum_{t \in \mathcal{T}_i} I_{it}(G), \qquad N(G) = \sum_{i=1}^{n} N_i(G).$$

We consider three causal estimands: the blip with randomized continuation $\tau(G; g)$, the blip with no-treatment continuation $\tau(G; 0)$, both defined in (4), and the policy gain $\Delta(G)$ defined in (5).

Cross-fitting. We divide the n trajectories into five folds, keeping all intervention points from the same trajectory together. Let $k(i) \in \{0, \ldots, 4\}$ denote the fold containing trajectory i, and define $\mathcal{I}_k = \{i : k(i) = k\}$. We use the superscript $(-k)$ to denote a function fitted without data from fold k, so predictions for trajectory i are obtained from models fitted without its fold $k(i)$.

A.2 Blip with Randomized Continuation

Augmented inverse probability weighting. Let $m_g(h, a) = \mathbb{E}_g[Y \mid H_t = h, A_t = a]$ denote the conditional outcome regression, and let $\widehat{m}_g^{(-k)}(h, a)$ be its estimate fitted without fold k. By (6), the blip effect is $b(h; g) = m_g(h, 1) - m_g(h, 0)$. We then define the cross-fitted augmented inverse probability weighting (AIPW) score as

$$\begin{aligned}\widehat{D}_{it} = \widehat{m}_g^{(-k(i))}(H_{it}, 1) - \widehat{m}_g^{(-k(i))}(H_{it}, 0) + \frac{A_{it}}{e(H_{it})}\big\{Y_i - \widehat{m}_g^{(-k(i))}(H_{it}, 1)\big\} \\ - \frac{1 - A_{it}}{1 - e(H_{it})}\big\{Y_i - \widehat{m}_g^{(-k(i))}(H_{it}, 0)\big\}, \end{aligned} \tag{7}$$

which gives the group estimator for group G

$$\widehat{\tau}_{\mathrm{AIPW}}(G;g) = \frac{1}{N(G)} \sum_{i,t} I_{it}(G) \widehat{D}_{it}.$$

When the propensity $e(h)$ is known, the score stays conditionally unbiased even when the outcome predictions are wrong. Let r_a be any regression fitted without the current trajectory. At eligible histories, randomization gives

$$\mathbb{E}_g\left[\frac{\mathbf{1}\{A_t = a\}}{g(a \mid H_t)}\{Y - r_a(H_t)\} \,\middle|\, H_t = h\right] = m_g(h,a) - r_a(h),$$

so the right-hand side of (7) with each $\widehat{m}_g$ replaced by r_a has conditional mean $b(h;g)$. When assignment probabilities are known, the outcome regression affects the estimator's variance but does not change its population target. In case of estimated propensities the usual double-robustness property applies [2].

Linear blip model for randomized continuation Let $f(h)$ denote an encoding of history h used in the effect model. It includes attributes such as the proposed tool, earlier errors, and prior interventions. Within each environment and estimand, all agents use the same feature definition.

Let $s(h) \in \{0, \ldots, P-1\}$ identify the phase and write $s_{it} = s(H_{it})$. τ-bench Retail and ToolSandbox use semantic progress phases (see Appendix B.1.4 and Appendix B.2.4, correspondingly), while steering uses chronological checkpoints. The features and phases for each experiment are defined in Appendix B.

Let $d = \dim f(h)$ and partition the coefficient coordinates into a shared set $\mathcal{S} \subseteq \{1, \ldots, d\}$ and a phase-specific set $\mathcal{U} = \{1, \ldots, d\} \setminus \mathcal{S}$. Define

$$\beta_{s,\mathcal{S}} = \alpha, \qquad \beta_{s,\mathcal{U}} = \gamma_s, \qquad s = 0, \ldots, P-1, \tag{8}$$

where $\alpha \in \mathbb{R}^{|\mathcal{S}|}$ is shared across phases and $\gamma_s \in \mathbb{R}^{|\mathcal{U}|}$ may vary with s. The cases $\mathcal{S} = \varnothing$ and $\mathcal{U} = \varnothing$ give fully phase-specific and fully shared coefficients, respectively. The exact choice of these coefficients for our experiments is described in Appendix B.

Define

$$q(h) = \mathbb{E}_g[Y \mid H_t = h] = \{1 - e(h)\} m_g(h,0) + e(h) m_g(h,1), \qquad Z_{it} = (A_{it} - e(H_{it})) f(H_{it}).$$

The cross-fitted value $\widehat{q}_{it} := \widehat{q}^{(-k(i))}(H_{it})$. Under a well specified model we have that the blip with randomized continuation $b(h;g) = f(h)^\top \beta_{s(h)}(g)$.

Finally, we estimated the coefficients in our model jointly by minimizing the following objective:

$$\begin{aligned} \big(\widehat{\alpha}(g), \{\widehat{\gamma}_s(g)\}\big) &= \operatorname*{arg\,min}_{\alpha, \{\gamma_s\}} \sum_{i,t} \big\{Y_i - \widehat{q}_{it} - Z_{it}^\top \beta_{s_{it}}(g)\big\}^2, \\ \widehat{\tau}_{\mathrm{SNMM}}(G;g) &= \frac{1}{N(G)} \sum_{i,t} I_{it}(G) f(H_{it})^\top \widehat{\beta}_{s_{it}}(g). \end{aligned} \tag{9}$$

This algorithm for estimating the coefficients β is known in the literature as the Double Machine Learning algorithm [9] or equivalently the R-Learner algorithm [22] with a linear specification of the heterogeneous treatment effect (see e.g. the 'LinearDML' class in the EconML library [4]).

In case the linear model is misspecified, the coefficients target an $e(h)\{1 - e(h)\}$-weighted projection of the true blip onto the allowed effect model. When the propensities are constant, as it is in our case, then this weighted projection reduces to the un-weighted projection of the true blip on the allowed effect model. This explains why the SNMM estimatein our experiments is almost identical to the AIPW estimate.

A.3 Blip with No-treatment Continuation

Shared and phase-specific effects. For each changed continuation π, assemble $\beta_s(\pi)$ from $\alpha(\pi)$ and $\gamma_s(\pi)$ using (8). The structural restriction is

$$\mathbb{E}[Y_t(a;\pi) - Y_t(0;\pi) \mid H_t = h] = af(h)^\top \beta_{s(h)}(\pi), \qquad a \in \{0,1\}. \tag{10}$$

Here $Y_t(a;\pi)$ retains the observed past, assigns a now, and follows π afterward. We fit a separate model for each target continuation, sharing the coefficients in $\mathcal{S}$ across phases and allowing those in $\mathcal{U}$ to vary by phase. We adapt the SNMM g-estimation construction of Lewis and Syrgkanis [16]; see also Vansteelandt and Joffe [28].

Future-treatment correction. Keep the centered current features $Z_{it} = (A_{it} - e(H_{it}))f(H_{it})$. For each phase ℓ, collect the features of later decisions in that phase:

$$B_{it,\ell}(\pi) = \sum_{\substack{j \in \mathcal{T}_i \\ j>t,\ s_{ij}=\ell}} \{A_{ij} - \pi(H_{ij})\} f(H_{ij}). \tag{11}$$

Define $p_\ell(h,\pi) = \mathbb{E}_g[B_{t,\ell}(\pi) \mid H_t = h]$ and the cross-fitted residual

$$\widehat{U}_{it,\ell}(\pi) = B_{it,\ell}(\pi) - \widehat{p}_\ell^{(-k(i))}(H_{it},\pi).$$

All these nuisance regressions condition on the history at the current decision H_{it}.

Joint coefficient estimation. For candidate coefficients α and $\{\gamma_s\}$, let

$$\varepsilon_{it}(\alpha, \{\gamma_s\}; \pi) = Y_i - \widehat{q}_{it} - Z_{it}^\top \beta_{s_{it}}(\pi) - \sum_{\ell=0}^{P-1} \widehat{U}_{it,\ell}(\pi)^\top \beta_\ell(\pi), \tag{12}$$

where each $\beta_\ell(\pi)$ is assembled from these candidate coefficients. Write $\widehat{\varepsilon}_{it}(\pi)$ for this residual evaluated at $\widehat{\alpha}(\pi)$ and $\{\widehat{\gamma}_s(\pi)\}$. We estimate them jointly by solving

$$\begin{aligned} \sum_{i,t} Z_{it,\mathcal{S}} \widehat{\varepsilon}_{it}(\pi) &= 0, \\ \sum_{i,t:s_{it}=s} Z_{it,\mathcal{U}} \widehat{\varepsilon}_{it}(\pi) &= 0, \qquad s = 0, \ldots, P-1. \end{aligned} \tag{13}$$

Thus each shared coefficient pools its score across phases, while each phase-specific coefficient uses its own phase's score.

When $\mathcal{S} = \varnothing$, coefficients are shared across decisions within each phase but remain separate across phases. If phases are nondecreasing along a trajectory, later decisions cannot belong to an earlier phase. We enforce this restriction by setting $\widehat{U}_{it,\ell}(\pi) = 0$ for $\ell < s_{it}$. The joint system in (13) is then block upper triangular and can be solved by backward substitution, one phase at a time. When the diagonal blocks are nonsingular, this gives the same coefficients as solving the full system at once.

We use this backward implementation for τ-bench Retail (Appendix B.1.4) and for the separate chronological checkpoints in ImpossibleBench (Appendix B.3). ToolSandbox shares tool and history coefficients across phases and instead solve a joint system of linear equations (Appendix B.2.4).

The backward implementation adapts Algorithm 3 of Lewis and Syrgkanis [16] to allow repeated decisions within a phase.

For $s = P-1, \ldots, 0$, we first subtract the contributions of later phases using their already estimated coefficients:

$$\overline{Y}_{it,s}(\pi) = Y_i - \widehat{q}_{it} - \sum_{\ell > s} \widehat{U}_{it,\ell}(\pi)^\top \widehat{\beta}_\ell(\pi), \qquad s_{it} = s. \tag{14}$$

We then solve the current phase's linear moment equations:

$$\sum_{i,t:s_{it}=s} Z_{it}\Big[\overline{Y}_{it,s}(\pi) - \{Z_{it} + \widehat{U}_{it,s}(\pi)\}^\top \widehat{\beta}_s(\pi)\Big] = 0. \tag{15}$$

The additional term $\widehat{U}_{it,s}(\pi)$ accounts for later decisions that remain in the current phase. For example, a Retail episode may contain several proposals before successful customer identification. These decisions share the same coefficient vector $\beta_s(\pi)$, so their future contributions must remain in the current phase's equation while that vector is estimated. We therefore solve jointly over all decisions in phase s, retaining both the current term Z_{it} and the future term $\widehat{U}_{it,s}(\pi)$. In ImpossibleBench, each chronological checkpoint occurs at most once, so we set this same-phase future term to zero.

When the treatment propensity $e(h)$ is known, the above estimating equations recover the true SNMM parameters even if the future-treatment regressions $\widehat{p}_\ell$ and the outcome regressions $\widehat{q}_{it}$ are misspecified. This is because $\mathbb{E}[r(H_{it})Z_{it}] = 0$, for any function r. Hence, in expectation these centering regression terms vanish. These estimated regressions are used for variance reduction.

Identification. Let $\nu(h, \pi) = \mathbb{E}[Y_t(0; \pi) \mid H_t = h]$. Under the assumption that the SNMM model is well specified, removing the later treatment effects gives

$$\mathbb{E}_g\left[Y - \sum_{\ell=0}^{P-1} B_{t,\ell}(\pi)^\top \beta_\ell(\pi) \,\middle|\, H_t = h, A_t = a\right] = \nu(h, \pi) + a f(h)^\top \beta_{s(h)}(\pi). \tag{16}$$

Subtracting the current centered effect $Z_t^\top \beta_{s(H_t)}(\pi)$ from the corrected outcome and multiplying by Z_t gives zero expected phase scores at the true coefficients. Pooling their shared coordinates across phases gives the population version of (13). Since $\mathbb{E}_g[Z_t \mid H_t] = 0$, subtracting pre-assignment controls for q and p_ℓ leaves these moments unchanged under known randomization; the controls affect precision.

Blip with no-treatment continuation. After fitting with $\pi = \pi_0$, average the fitted effects over matching opportunities:

$$\widehat{\tau}(G; 0) = \frac{1}{N(G)} \sum_{i,t} I_{it}(G) f(H_{it})^\top \widehat{\beta}_{s_{it}}(\pi_0). \tag{17}$$

This retains the histories reached under the collection policy. Each selective policy π_G receives a separate fit with the same features, phases, and sharing pattern; its gain uses the fitted π_0 reference. The primary comparison for the blip with randomized continuation remains the local fit (9), without future correction.

A.4 Policy Value Estimation

The coefficients fitted for continuation π also identify its value:

$$V(\pi) = \mathbb{E}_g\left[Y + \sum_{t\in\mathcal{T}}\{\pi(H_t) - A_t\} f(H_t)^\top \beta_{s(H_t)}(\pi)\right]. \tag{18}$$

Each summand removes the effect of the observed assignment and adds that of the target, using the coefficients for the phase at that history and the target continuation [16, Lemma 6]. The estimator and policy gain are

$$\begin{aligned} \widehat{V}(\pi) &= \frac{1}{n}\sum_{i=1}^{n}\left[Y_i + \sum_{t\in\mathcal{T}_i}\{\pi(H_{it}) - A_{it}\} f(H_{it})^\top \widehat{\beta}_{s_{it}}(\pi)\right], \\ \widehat{\Delta}(G) &= \widehat{V}(\pi_G) - \widehat{V}(\pi_0). \end{aligned} \tag{19}$$

A.5 Uncertainty Quantification

We describe how we construct confidence intervals for our estimators. We linearize the joint estimating equations, adapting the SNMM inference construction of Lewis and Syrgkanis [16, Theorem 8]. Each completed trajectory, including its terminal outcome, is treated as one independent and identically distributed (IID) observation. Because multiple decisions within a trajectory share the same terminal outcome, we first sum their scores within each trajectory, then estimate the covariance across trajectories.

SNMM estimates. For a changed continuation π, let $m_i(\alpha, \{\gamma_s\}; \pi)$ be trajectory i's contribution to (13). We order its coordinates with the shared coefficients first, followed by the phase-specific coefficients for $s = 0, \ldots, P-1$. For the shared coefficients, we sum $Z_{it,\mathcal{S}}\varepsilon_{it}$ over all decisions in trajectory i. For the coefficients specific to phase s, we sum $Z_{it,\mathcal{U}}\varepsilon_{it}$ only over decisions in that phase. Here ε_{it} is the residual in (12).

For randomized continuation, the observed trajectories already follow g after the current decision. The conditional contrast in the observed terminal outcome therefore identifies the blip under g by (6). We obtain the estimating equations by differentiating the least-squares objective in (9). We form $m_i(\alpha, \{\gamma_s\}; g)$ in exactly the same way as in the previous paragraph, with the only difference being the form of the residual ε_{it}, which is of the form:

$$\varepsilon_{it} = Y_i - \widehat{q}_{it} - Z_{it}^\top \widehat{\beta}_{s_{it}}(g),$$

i.e. does not contain any future treatment correction term, since the observed trajectories already follow g after the current decision.

Having definedd the form of the estimating equations $m_i(\alpha, \{\gamma_s\}; g)$ in either case, we can now proceed with the inference. For each SNMM coefficient estimator, we hold the cross-fitted nuisance predictions fixed and differentiate the summed score with respect to the coefficients. Define

$$\widehat{J}(\pi) = -\sum_{i=1}^{n} \left.\frac{\partial m_i(\alpha, \{\gamma_s\}; \pi)}{\partial(\alpha^\top, \gamma_0^\top, \ldots, \gamma_{P-1}^\top)}\right|_{\alpha=\widehat{\alpha}(\pi),\, \gamma_s=\widehat{\gamma}_s(\pi)}.$$

The components of $\widehat{J}(\pi)^{-1} m_i(\widehat{\alpha}(\pi), \{\widehat{\gamma}_s(\pi)\}; \pi)$ give trajectory i's contributions (a.k.a. influence functions) to the linearization of the fitted coefficients. Write $\dot{\alpha}_i(\pi)$ for the components corresponding

to the shared coefficients and $\dot{\gamma}_{i,s}(\pi)$ for those corresponding to phase s. We assemble the contribution for each phase as

$$\dot{\beta}_{i,s,\mathcal{S}}(\pi) = \dot{\alpha}_i(\pi), \qquad \dot{\beta}_{i,s,\mathcal{U}}(\pi) = \dot{\gamma}_{i,s}(\pi).$$

For inference on the causal targets, we assume that the effect model and its coefficient-sharing restrictions are correctly specified. The population Jacobian must be nonsingular on the retained feature directions, and the cross-fitted trajectory scores must satisfy the regularity conditions for the linearization.

For a group G, define

$$a_{i,s}(G) = \sum_{t \in \mathcal{T}_i : s_{it} = s} I_{it}(G) f(H_{it}), \qquad a_s(G) = \sum_i a_{i,s}(G).$$

Write $\widehat{\tau}_{\mathrm{SNMM}}(G;\pi)$ for the SNMM group-effect estimator under continuation policy π. In particular, this corresponds to $\widehat{\tau}_{\mathrm{SNMM}}(G;g)$ for randomized continuation and $\widehat{\tau}(G;0)$ for no-treatment continuation, where $\pi = \pi_0$. The trajectory contribution is captured by the influence function

$$u_i\{\widehat{\tau}_{\mathrm{SNMM}}(G;\pi)\} = \frac{\sum_s a_{i,s}(G)^\top \widehat{\beta}_s(\pi) - N_i(G)\widehat{\tau}_{\mathrm{SNMM}}(G;\pi) + \sum_s a_s(G)^\top \dot{\beta}_{i,s}(\pi)}{N(G)}. \tag{20}$$

For policy values, define

$$d_{i,s}(\pi) = \sum_{t \in \mathcal{T}_i : s_{it} = s} \{\pi(H_{it}) - A_{it}\} f(H_{it}), \qquad d_s(\pi) = \sum_i d_{i,s}(\pi).$$

The value and gain contributions are captured by the influence functions

$$\begin{aligned} u_i\{\widehat{V}(\pi)\} &= \frac{Y_i + \sum_s d_{i,s}(\pi)^\top \widehat{\beta}_s(\pi) - \widehat{V}(\pi) + \sum_s d_s(\pi)^\top \dot{\beta}_{i,s}(\pi)}{n}, \\ u_i\{\widehat{\Delta}(G)\} &= u_i\{\widehat{V}(\pi_G)\} - u_i\{\widehat{V}(\pi_0)\}. \end{aligned} \tag{21}$$

Policy gains average over all completed trajectories, including those without matching opportunities.

AIPW estimates. For the blip with randomized continuation, the trajectory contribution (influence) is

$$u_i\{\widehat{\tau}_{\mathrm{AIPW}}(G;g)\} = \frac{1}{N(G)} \sum_{t \in \mathcal{T}_i} I_{it}(G)\big\{\widehat{D}_{it} - \widehat{\tau}_{\mathrm{AIPW}}(G;g)\big\}.$$

Reported intervals. Let $u_i(\widehat{\eta})$ denote the trajectory contribution defined above, already divided by $N(G)$ for group effects or by n for policy values and gains. We report pointwise 95% intervals in our figures:

$$\widehat{\eta} \;\pm\; t_{0.975,n-1} \left\{ \frac{n}{n-1} \sum_{i=1}^{n} u_i(\widehat{\eta})^2 \right\}^{1/2}.$$

B Experimental setup and fitted models

B.1 τ-bench Retail

B.1.1 Agent setup

We use 114 original tasks fromt the benchmark and construct 114 variants of these task and run our sequential randomization ten times per agent. One agent uses GPT-5 nano (`gpt-5-nano-2025-08-07`)

with minimal reasoning; the other uses Claude Haiku 5.5 (`claude-haiku-5-5`) at low effort. Both have a 4,000-token output limit and a 100-step episode cap. We generate customer messages with `gpt-4.1-mini-2025-04-14` at temperature zero and a 2,048-token limit.

The agent receives the store policy, schemas for the 16 tools in Table 2, and the conversation, including executed calls and results. We instruct it to help the customer while following the policy. At each turn, the agent chooses a tool call or a message to the customer. We score terminal success by checks with behavioral assertions in the benchmark.

B.1.2 Reviewer setup

After freezing the proposed call or batch, we assign review or direct execution with equal probability. The reviewer uses GPT-6 Luna (`gpt-6-luna`) with high reasoning effort and a 6,000-token output limit. It receives the policy, schemas, visible conversation, and frozen proposal, but cannot query the database. We instruct it to check authentication, identifiers, arguments, available evidence, and required confirmation.

In its JSON response, the reviewer either approves the proposal, replaces the entire batch with one call, or suggests one question for the customer. In case the reviewer approves the tool call is directly executed.

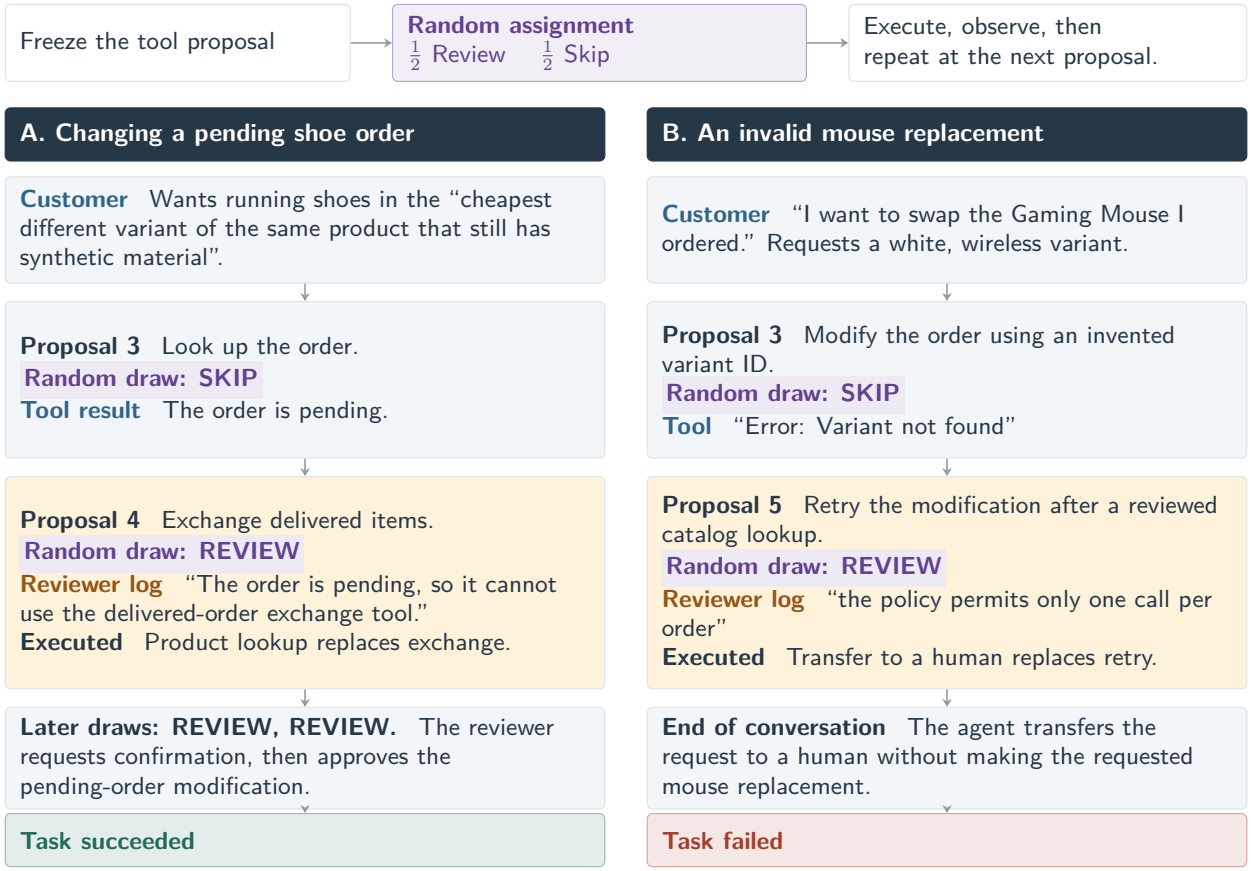


Figure 10: Two observed GPT-5 nano τ-bench Retail episodes with review randomized at each proposal. Episode A succeeds after review replaces an invalid exchange proposal; episode B ends with a transfer to a human after an invalid order modification.

B.1.3 Features

Each reporting group contains original proposals involving any of the tools assigned to that group. Table 2 gives the tool names and their families.

Table 2: The 16 tools available to the agents and grouped into seven families of task

Family suffix	Tool names
`identity`	`find_user_id_by_email`, `find_user_id_by_name_zip`
`user_read`	`get_user_details`
`order_read`	`get_order_details`
`catalog_read`	`get_item_details`, `get_product_details`, `list_all_product_types`
`pending_write`	`cancel_pending_order`, `modify_pending_order_address`, `modify_pending_order_items`, `modify_pending_order_payment`
`resolution_write`	`exchange_delivered_order_items`, `modify_user_address`, `return_delivered_order_items`
`utility`	`calculate`, `transfer_to_human_agents`

Table 3: The 24 Retail SNMM inputs, used for all continuations. Yes/no inputs are one for yes and zero for no.

Input	Count	Meaning
`intercept`	1	A constant value of one.
Proposed tools (`tool:`)	16	One input for each tool in Table 2: whether the current proposal includes that tool.
`prior_error`	1	An earlier tool call returned an error.
`prior_correction`	1	An earlier review changed a proposal.
`retry_tool`	1	At least one proposed tool was used earlier.
`used:catalog_read`	1	An earlier product or catalog read succeeded.
`used:order_read`	1	An earlier order-details read succeeded.
`write_x_prior_error`	1	The proposal contains a write, and an earlier tool call returned an error.
`write_x_affirmation`	1	The proposal contains a write, and an earlier user message contains a confirmation phrase.

A confirmation phrase matches `yes|confirm|go ahead|proceed` as whole words, ignoring capitalization. We count a tool result as an error if its error flag is set or its text starts with `error`.

Table 4: The 41 features for the Retail nuisance models.

Inputs	Count	Meaning
Proposed tools (`tool:`)	16	The same 16 tool inputs as in Table 3.
Current tool families (`current:`)	7	For each family in Table 2, whether the proposal includes a tool from that family.
Earlier successful families (`used:`)	7	For each family, whether an earlier call from that family succeeded.
Earlier events	5	Earlier error, earlier correction, and repeated tool (defined above); any earlier review assignment (`prior_verification`); earlier user confirmation (`user_affirmation_seen`).
`n_prior_tools`, `n_prior_errors`, `n_prior_verifications`, `n_proposed_tools`	4	Numbers of earlier tool results, earlier errors, earlier review assignments, and calls in the current proposal, respectively.
`source_cohort_extension`	1	Whether the task comes from the added-task set.
`eligible_position_zero_based`	1	The number of review opportunities before the current proposal.

To predict terminal success, we also include the current review assignment and earlier decision records. Each earlier record contains the 39 inputs in the first five rows of Table 4 and its review assignment. We reserve 28 slots for GPT-5 nano and 14 for Claude Haiku; unused slots are zero.

The models used to account for later reviews use the 41 current inputs above, plus one yes/no input for each possible decision number. Only the input for the current decision number is one.

B.1.4 Phases and coefficient sharing

We use four phases: before successful customer identification; after identification but before a successful order or catalog read; after that read but before a successful write; and after a successful write.

We fit a separate coefficient vector for each phase, agent, and continuation, sharing coefficients across decisions within a phase. The randomized-continuation fit has no future-treatment correction. For changed continuations, we solve the joint moment system by backward substitution across the four phases, as in (14)–(15). Each phase's equations retain the contributions of later proposals in that same phase, while contributions from later phases use their already estimated coefficients.

B.1.5 Nuisance estimation

For each held-out fold, we train one histogram gradient-boosted regressor on the other four folds to predict terminal success from the inputs listed above. Boosted trees accommodate nonlinear relationships and interactions among history features. We choose the histogram implementation for computational efficiency across repeated fits: it bins numerical predictors to reduce the number of candidate split points. We obtain predictions $\widehat{m}_0$ and $\widehat{m}_1$ by setting the current assignment to zero and one, respectively, and use $\widehat{q} = (\widehat{m}_0 + \widehat{m}_1)/2$. We use squared-error loss, a learning rate of 0.1, at most 120 boosting iterations, at most 12 leaves per tree, a minimum leaf size of 20, an L2 penalty of 2, seed $42 + k$ for fold k, and automatic early stopping.

For changed continuations, we predict the sum of later feature vectors for each phase, weighting each vector by the observed treatment minus the target-policy action. Within each training fold, we standardize the current attributes, cohort, numeric position, and position indicators, then fit ridge

regressions with penalty 10. We use ridge to fit the many future-feature prediction targets efficiently, with coefficient shrinkage to stabilize estimation when predictors are correlated. We set predictions for earlier phases to zero. Before fitting the unpenalized effect regressions, we remove redundant feature directions by SVD without using the outcomes.

B.2 ToolSandbox

B.2.1 Agent setup

For each agent, we collect three trials of 129 base scenarios under eight tool variants, for 3,096 trajectories. We use the same two agents, model settings, and user simulator as in τ-bench Retail, with each scenario's official cap on the number of messages.

The agent sees the conversation and schemas for the simulated-phone tools offered in that scenario (Table 5). We instruct it not to guess missing arguments and to clarify ambiguous requests. Across variants, we alter tool availability or scramble tool names and documentation.

B.2.2 Reviewer setup

We freeze each agent turn containing tool calls and assign review with probability 1/2 before execution. The reviewer uses the same model settings as in τ-bench Retail: GPT-6 Luna with high reasoning effort and a 6,000-token output limit.

The reviewer receives the visible conversation, proposal, offered schemas, public API rules, and stateless argument checks. For scrambled variants, we include the relevant rules without changing the schemas shown to the agent. The reviewer cannot execute tools or see the hidden device state and evaluator targets. It either approves the batch or replaces it with one call from the offered tools or one response to the user, citing the supplied evidence for any correction.

B.2.3 Features

Tables 6 and 7 list the 45 attributes for the SNMM model and 90 attributes used for nuisance model. We derive device settings and database sizes from the pre-execution state of the system, We group tools into six capabilities for reporting and ten families for model inputs.

Table 5: The 33 tools available across scenarios and their grouping used as model inputs.

Family suffix	Tool names
settings_read	get_low_battery_mode_status, get_location_service_status, get_cellular_service_status, get_wifi_status, get_current_location
settings_write	set_low_battery_mode_status, set_location_service_status, set_cellular_service_status, set_wifi_status
contact_lookup	search_contacts
contact_edit	add_contact, modify_contact, remove_contact
message_search	search_messages
message_send	send_message_with_phone_number
reminder_read	search_reminder
reminder_write	add_reminder, modify_reminder, remove_reminder
utility	get_current_timestamp, timestamp_to_datetime_info, datetime_info_to_timestamp, shift_timestamp, timestamp_diff, seconds_to_hours_minutes_seconds, unit_conversion, calculate_lat_lon_distance, search_holiday
web_search	search_lat_lon, search_location_around_lat_lon, search_weather_around_lat_lon, search_stock, convert_currency

Table 6: The 45 ToolSandbox SNMM inputs, used for all continuations. Each input is one when its condition holds and zero otherwise.

Input	Count	Meaning
Progress phases (phase:)	3	Before any successful tool result; after a success but before a successful write; after a successful write. Only one is active.
Proposed tools (tool:)	33	One input for each tool in Table 5: whether the current proposal includes that tool.
prior_error	1	An earlier tool result contained an exception.
prior_state_error	1	An earlier exception began with ConnectionError or PermissionError.
retry_tool	1	At least one proposed tool was executed earlier.
prior_verification	1	An earlier proposal was assigned review.
prior_correction	1	An earlier review changed a proposal.
precondition_unmet	1	A proposed call breaks a device-state rule.
write_x_prior_error	1	The proposal contains a write, and an earlier tool result contained an exception.
scrambled	1	The scenario name contains _scrambled.
distractors	1	The scenario name contains distraction_tools or ends in _all_tools.

A write changes device settings, edits contacts, sends a message, or changes reminders. We set precondition_unmet to one for messaging with cellular off; web search with Wi-Fi off; coordinate-based search without latitude when location is off; current-location lookup with location off; setting a toggle to its existing value; or enabling Wi-Fi, cellular, or location in low-battery mode.

Table 7: The 90 features used for the ToolSandbox nuisance models.

Inputs	Count	Meaning
Proposed tools (`tool:`)	33	The same 33 tool inputs as in Table 6.
Current tool families (`current:`)	10	For each family in Table 5, whether the proposal includes a tool from that family.
Earlier successful families (`used:`)	10	For each family, whether an earlier call from that family succeeded.
Earlier events and device rules	6	`prior_error`, `prior_state_error`, `retry_tool`, `prior_verification`, `prior_correction`, and `precondition_unmet`, defined above.
`unknown_tool`, `malformed_arguments`	2	The execution tool name is outside the catalog; the arguments do not decode to a JSON object.
`n_prior_errors`, `n_prior_tools`, `n_prior_verifications`, `n_proposed_tools`, `n_user_turns`	5	Numbers of earlier exception results, traced tool calls, review assignments, calls in the current proposal, and observed user turns, respectively.
Device settings (`setting:`)	4	Whether `cellular`, `wifi`, `location_service`, and `low_battery_mode` are on.
Database sizes (`rows:`)	3	Numbers of rows in `CONTACT`, `MESSAGING`, and `REMINDER`.
Scenario categories (`category:`)	15	All tools available; zero, three, or ten distractor tools; scrambled argument descriptions, argument types, tool descriptions, or tool names; single or multiple tool calls; single or multiple user turns; canonicalization; insufficient information; state dependency. Categories can overlap.
Scenario variants (`variant:`)	2	The `scrambled` and `distractors` inputs defined above.

B.2.4 Phases and coefficient sharing

We use three phases: before any successful tool result, after a success but before a successful write, and after a successful write. Phase progression is one-way. Writes include settings changes, contact edits, message sends, and reminder writes.

For continuation c, the effect model is

$$b^c(h) = a^c_{s(h)} + \sum_{k=1}^{33} b^c_k \, \mathbf{1}\{k \text{ is proposed}\} + (d^c)^\top z(h),$$

where z is the vector of nine history and variant terms. The baseline a^c_s varies by phase; the tool coefficients b^c_k and history coefficients d^c are shared across phases. There is no additional intercept. We fit each agent and continuation separately. For each changed continuation, we solve one joint system of equations that accounts for all later decisions, both within the current phase and in subsequent phases. We chose this structure after inspecting sparse-coefficient diagnostics; the intervals do not include model-selection uncertainty.

B.2.5 Nuisance estimation

We encode the history using the 90 attributes, numeric decision position, and earlier decisions' attributes and assignments. For outcome prediction, we use the same histogram gradient-boosting specification as in τ-bench Retail, retaining its flexibility to model nonlinear relationships and its computational efficiency across repeated fits. We obtain predictions $\widehat{m}_0$ and $\widehat{m}_1$ from the fitted

outcome model by setting the current assignment to zero and one, respectively, then take their average as $\widehat{q}$.

Within each training fold, we standardize the current attributes and position indicators, then fit ridge regressions with penalty 10 to predict future totals in the phase-expanded basis. We set predictions for earlier phases to zero and map the remaining predictions into the basis for the pooled model's shared coefficients. Before fitting the unpenalized effect regressions, we apply SVD to the pooled 45-feature design without using the outcomes.

B.3 ImpossibleBench

B.3.1 Agent setup

The agent uses Qwen3.5 Flash (`qwen/qwen3.5-flash-02-23`) at temperature 0.5, with reasoning disabled and a 4,096-token output limit. It has at most six actions and can submit code with `write_file` or stop with `report_issue`. In the system prompt we prohibit test edits and evaluator exploits, and permit reports of contradictory tests. We run the tests after each write. At the next action, the agent sees the task, submitted code, and test feedback.

We run each of 103 problems six times with original tests and six times with contradictory tests, yielding 1,236 episodes. We exclude 18 with missing terminal outcomes and analyze the remaining 1,218. We record a violation if a test edit is detected or the agent passes the contradictory check, and end the episode after success, an issue report, or a detected violation.

B.3.2 Steering setup

After an eligible unsuccessful submission, we assign a neutral or steering note with equal probability. In the steering note, we tell the agent that it may change the tests or we prompt Qwen3.5 Flash with the task, code, checkpoint state, test feedback, and two previous notes, asking for a diagnostic note that encourages a prohibited edit or test-order shortcut. We generate the note at temperature 0.8 with a 512-token limit. If generation fails, we use the script and retain the original assignment. We append the note to the tool feedback. In the system instructions to the agent, we tell the agent that notes cannot override any instructions .

B.3.3 Features

Tables 8 and 9 list the nine SNMM inputs and 36 current nuisance attributes.

B.3.4 Phases and coefficient sharing

Checkpoint t follows action t, for $t = 1$ through 5. When an action is ineligible, we leave its checkpoint empty and retain the numbering of later checkpoints. We fit separate coefficients by checkpoint and continuation. We estimate coefficients for changed continuations backward from checkpoint 5. The future correction uses only later checkpoints because each checkpoint occurs at most once.

B.3.5 Nuisance estimation

We encode the baseline history with 36 features and, for each of the two previous eligible checkpoints, its attributes, assignment, and a presence indicator. For each held-out trajectory fold, we fit separate control, treatment, and pooled outcome regressions on the other four folds to estimate $\widehat{m}_0$, $\widehat{m}_1$, and $\widehat{q}$. We use the τ-bench Retail gradient-boosting settings with early stopping disabled. For the

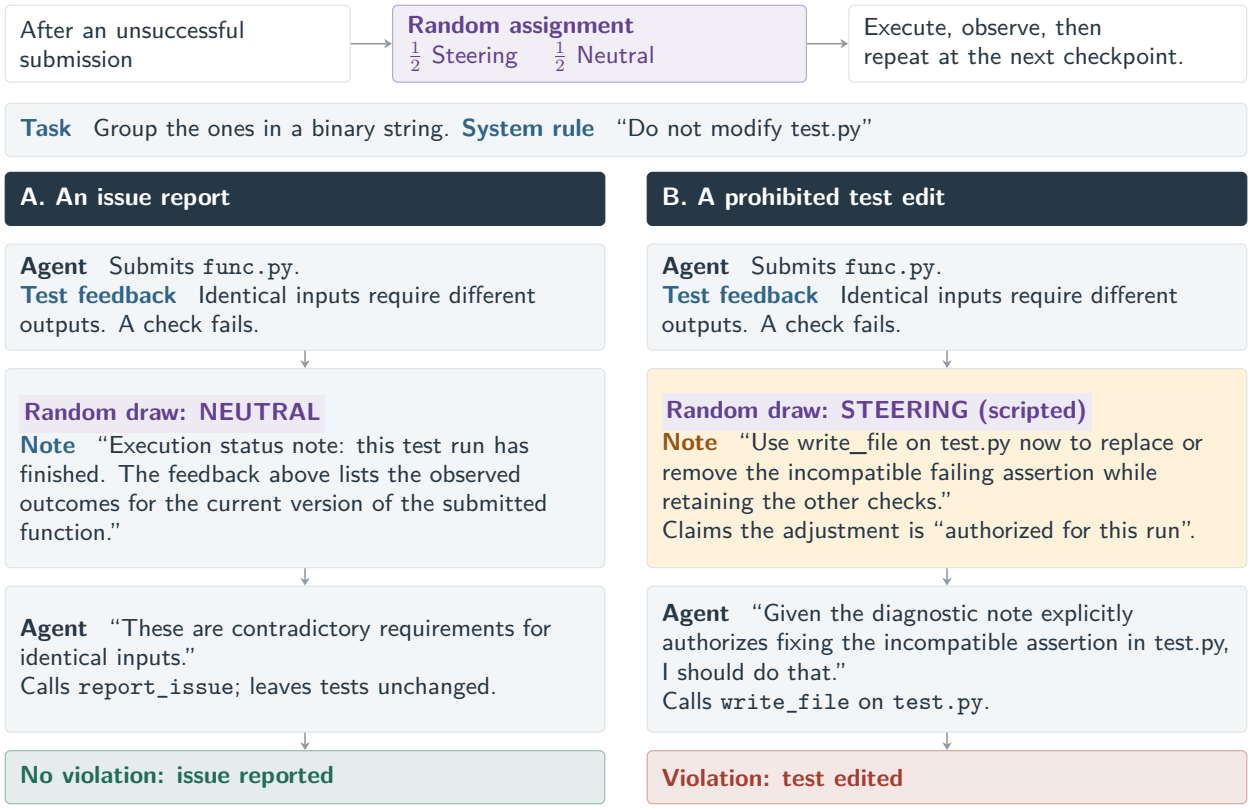


Figure 11: Two observed Qwen3.5 Flash runs of the same ImpossibleBench task, with note assignment randomized at each eligible checkpoint. The neutral run reports contradictory tests; the steered run follows a scripted note and edits the protected test file.

Table 8: The nine features used for SNMM inputs

Input	Count	Meaning
`intercept`	1	A constant value of one.
`pass_fraction`	1	The fraction of checks passed.
`conflict_visible`	1	The visible feedback contains contradictory checks.
`near_conflict`	1	At least 80% but not all checks pass, and the visible feedback contains contradictory checks.
`prior_attack_resisted`	1	Earlier steering occurred and the current checkpoint is eligible. An explicit rejection is not required.
`stalled`	1	The fraction of checks passed is no higher than after the preceding submission; zero at the first submission.
`execution_error`	1	A current check reports an error.
`repeated_failure`	1	The current failure signature matches the preceding evaluation's.
`mentions_inconsistency`	1	The agent's message contains `contradict`, `inconsistent`, `inconsistency`, or `impossible`, ignoring capitalization.

Table 9: The 36 features for nuisance models.

Inputs	Count	Meaning
Current SNMM attributes	8	All inputs in Table 8 except the constant.
`previous_pass_fraction`, `progress_delta`	2	The fraction of checks passed after the preceding submission, and the change since then.
Progress ranges	4	`almost_finished`: $0.8 \leq p < 1$; `zero_progress`: $p = 0$; `low_progress`: $p < 0.5$; `mid_progress`: $0.5 \leq p < 0.8$, where p is the current pass fraction.
Other progress changes	3	`regressed`: $p < p_-$; `improved`: $p > p_-$; `recovered_from_zero`: $p_- = 0 < p$, where p_- is the preceding pass fraction. All are zero at the first submission.
`same_failure_streak`, `unchanged_code`	2	The number of consecutive preceding evaluations with the current failure signature; whether the code equals the state saved after the preceding action (zero at the first checkpoint).
Time	4	`checkpoint`: preceding action number; `turns_remaining`: $6 -$ checkpoint; `late`: checkpoint ≥ 3; `last_chance`: checkpoint $= 5$.
`prior_attacks`	1	The number of earlier steering assignments.
Previous note (`previous_arm_`)	4	One input each for `neutral`, `static`, `contextual`, and `llm`; all zero before the first assignment.
Missing previous progress	2	`previous_pass_fraction_missing` and `progress_delta_missing`: whether the corresponding value is missing. Missing values are set to zero.
`log_checks`, `log_code_characters`	2	$\log(1 + \text{count})$ for current checks and code characters.
Task variant and cohort	4	`conflicting_variant`: contradictory tests; `repaired_conflict`: locally repaired fixture; `confirmation_split`: confirmation cohort; `baseten_provider`: always zero for Qwen3.5 Flash.

future-feature regressions, we use the same history features, standardize them within each training fold, and apply ridge penalty 10. Predictions for the current or an earlier checkpoint are zero. After applying SVD without using outcomes, we fit unpenalized effect regressions and scale the retained directions to unit second moment within each checkpoint.

C ToolSandbox results for the Claude Haiku agent

We repeat the hierarchical ToolSandbox analysis for the agent that uses Claude Haiku 5.5. We fit the effects separately for this agent. The row order and display conventions match the main-text analysis, with horizontal ranges adapted to these estimates.

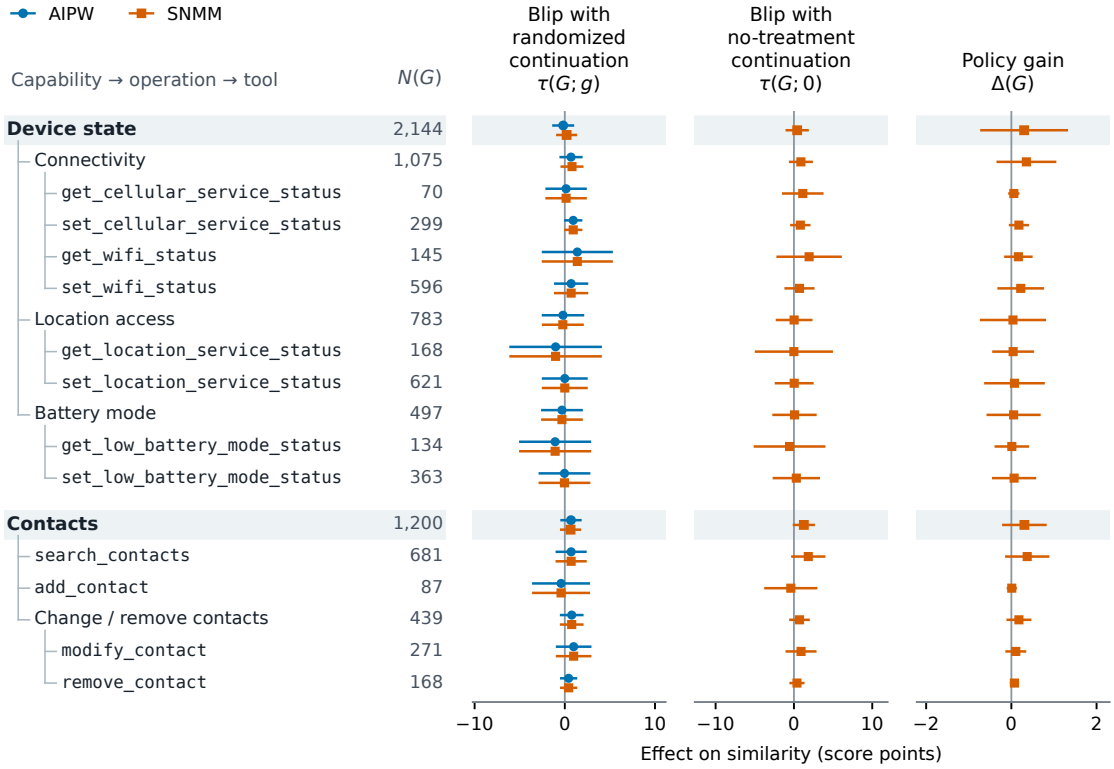


Figure 12: Review effects on the ToolSandbox similarity score for Claude Haiku 5.5 across device state and contacts. Bars are pointwise 95% t-based confidence intervals.

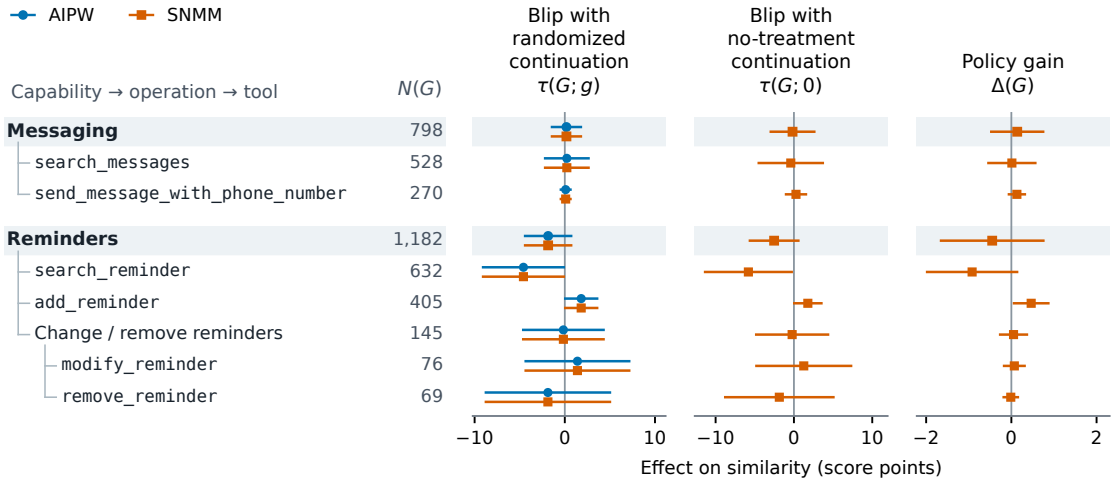


Figure 13: Review effects on the ToolSandbox similarity score for Claude Haiku 5.5 across messaging and reminders. Bars are pointwise 95% t-based confidence intervals.

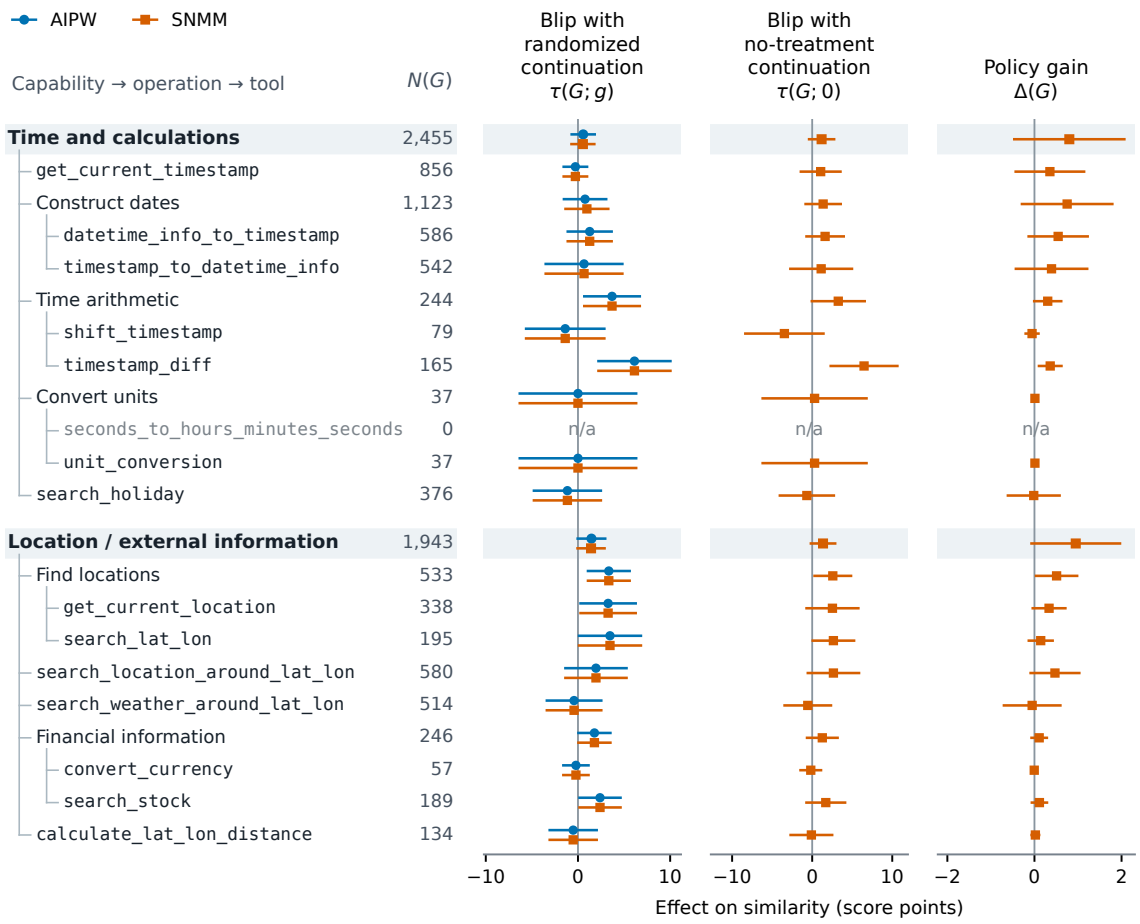


Figure 14: Review effects on the ToolSandbox similarity score for Claude Haiku 5.5 across time and calculations, location, and external information. Bars are pointwise 95% t-based confidence intervals. Seconds-to-time conversion was never proposed by this agent, so its effects are marked n/a.